



BM Công nghệ phần mềm – Khoa CNTT
se.nuce.edu.vn

Cấu trúc cây (Tiếp)

Chương 1. Tree

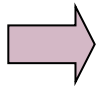
- 1.1. Các khái niệm cơ bản
- 1.2. Phép duyệt cây và biểu diễn cây
- 1.3. Cây nhị phân và cây nhị phân tìm kiếm
- 1.4. Cây AVL
- 1.5. Heap

Chương 1. Tree

1.1. Các khái niệm cơ bản

1.2. Phép duyệt cây và biểu diễn cây

1.3. Cây nhị phân và cây nhị phân tìm kiếm



1.4. Cây AVL

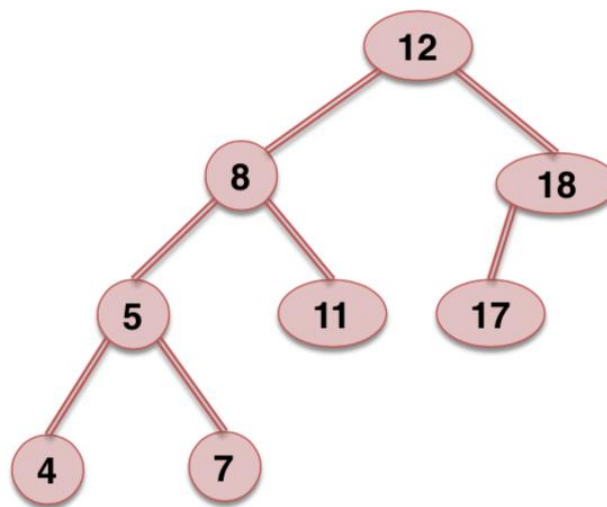
1.5. Heap

1.4. Cây AVL

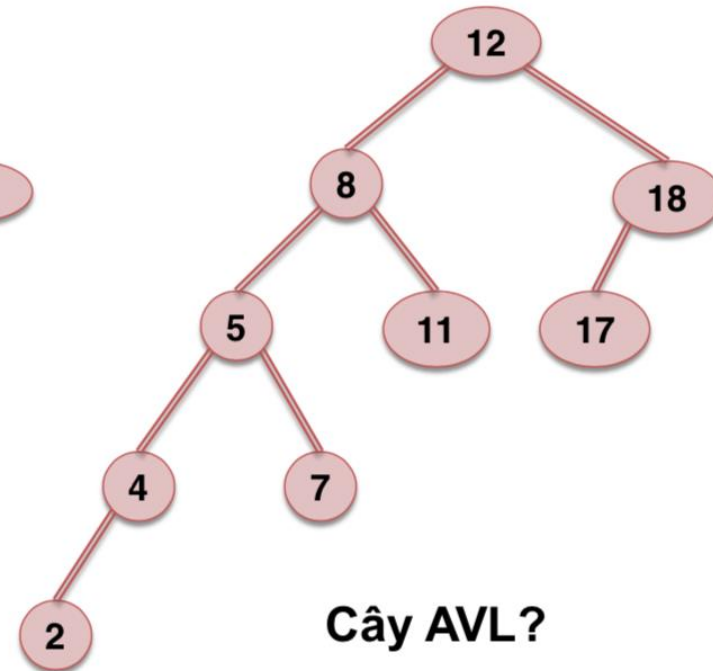
- Do G.M. Adelsen Velskii và E.M. Lendis đưa ra vào năm 1962, đặt tên là cây AVL.

1.4.1. Định nghĩa

- Định nghĩa : Cây cân bằng AVL là cây nhị phân tìm kiếm mà tại mỗi đỉnh của cây, độ cao của cây con trái và cây con phải không chênh lệch quá 1.
- Ví dụ



Cây AVL?



Cây AVL?

1.4.2. Xây dựng cây cân bằng

- Việc xây dựng cây cân bằng dựa trên cây nhị phân tìm kiếm, chỉ bổ sung thêm 1 giá trị cho biết sự cân bằng của các cây con như thế nào.
- Cách làm gợi ý

```
struct NODE
{
    Data key;
    NODE *pLeft;
    NODE *pRight;
    int bal;
};
```

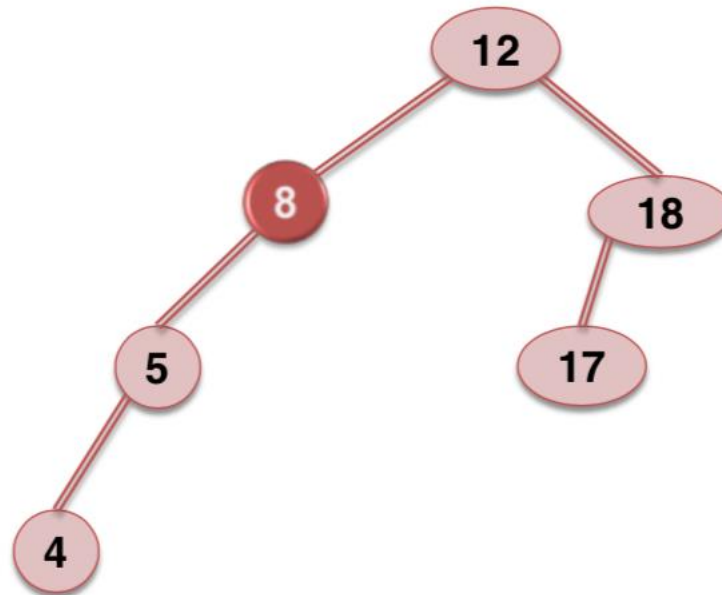
- Trong đó giá trị bal (balance, cân bằng) có thể là: 0: cân bằng; 1: lệch trái; 2: lệch phải

1.4.3. Các trường hợp mất cân bằng

- Mất cân bằng trái-trái (L-L)
- Mất cân bằng trái-phải(L-R)
- Mất cân bằng phải-phải(R-R)
- Mất cân bằng phải-trái(R-L)

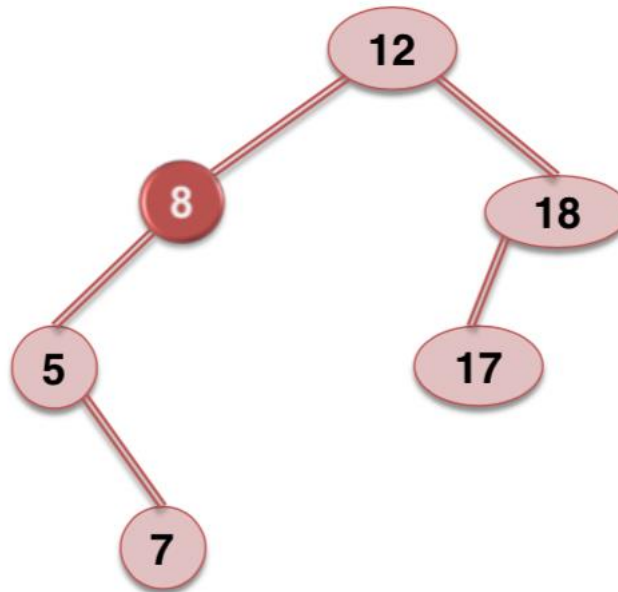
1.4.3. Các trường hợp mất cân bằng

- Mất cân bằng trái-trái (L-L)



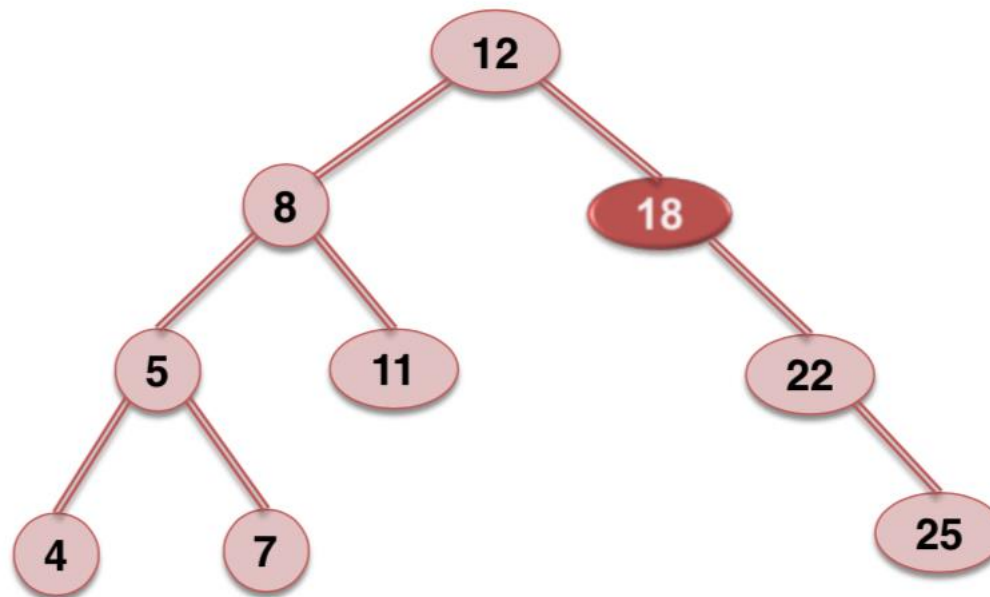
1.4.3. Các trường hợp mất cân bằng

- Mất cân bằng trái-phải (L-R)



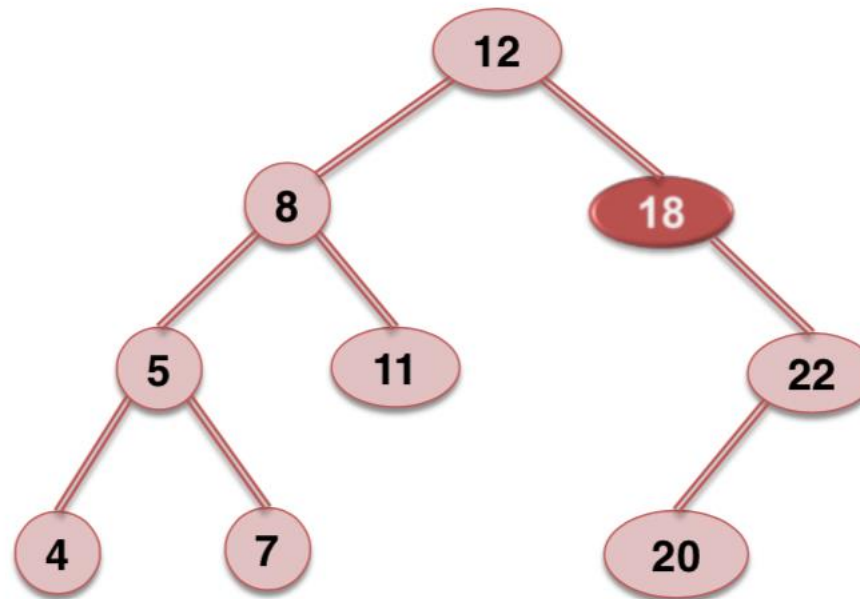
1.4.3. Các trường hợp mất cân bằng

- Mất cân bằng phải-phải (R-R)



1.4.3. Các trường hợp mất cân bằng

- Mất cân bằng phải-trái (R-L)

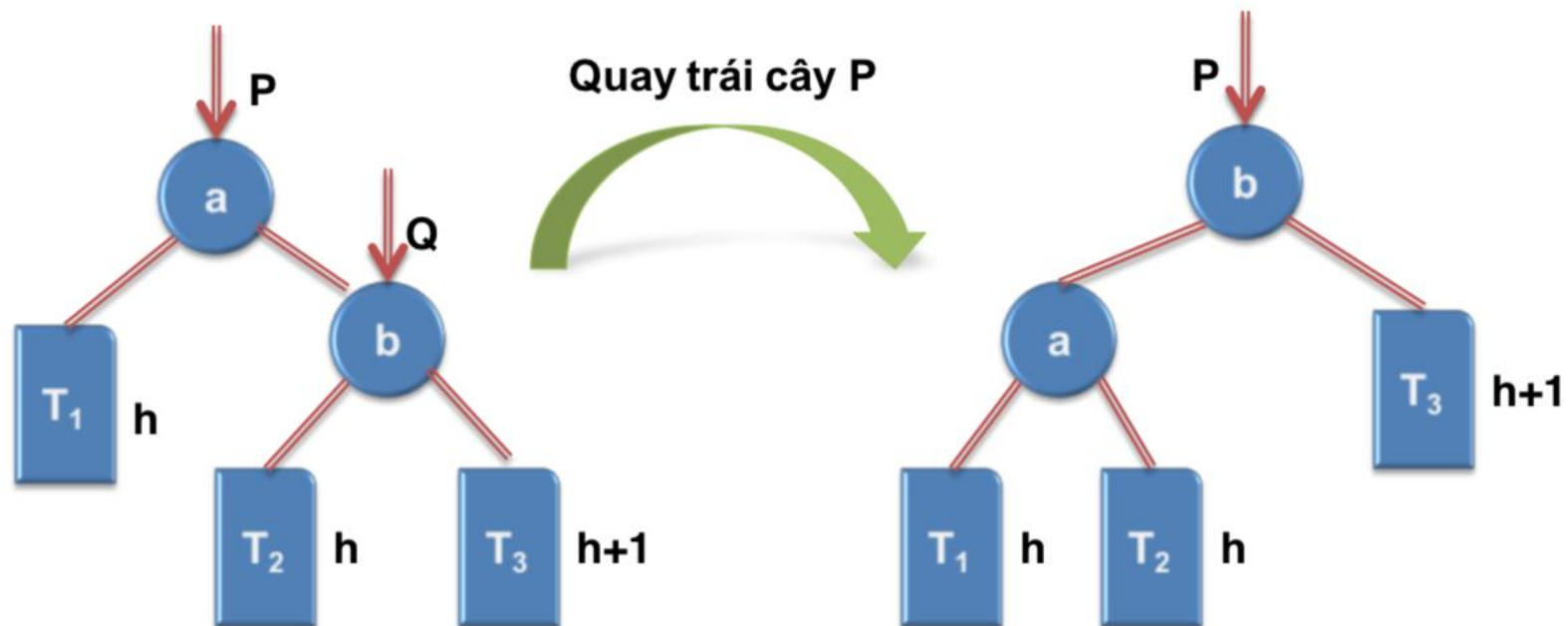


1.4.4. Xử lý mất cân bằng

- Giả sử tại một node cây xảy ra mất cân bằng bên phải (cây con phải chênh lệch với cây con trái hơn một đơn vị):
 - Mất cân bằng phải-phải (RR)
 - Quay trái
 - Mất cân bằng phải-trái (R-L)
 - Quay phải
 - Quay trái

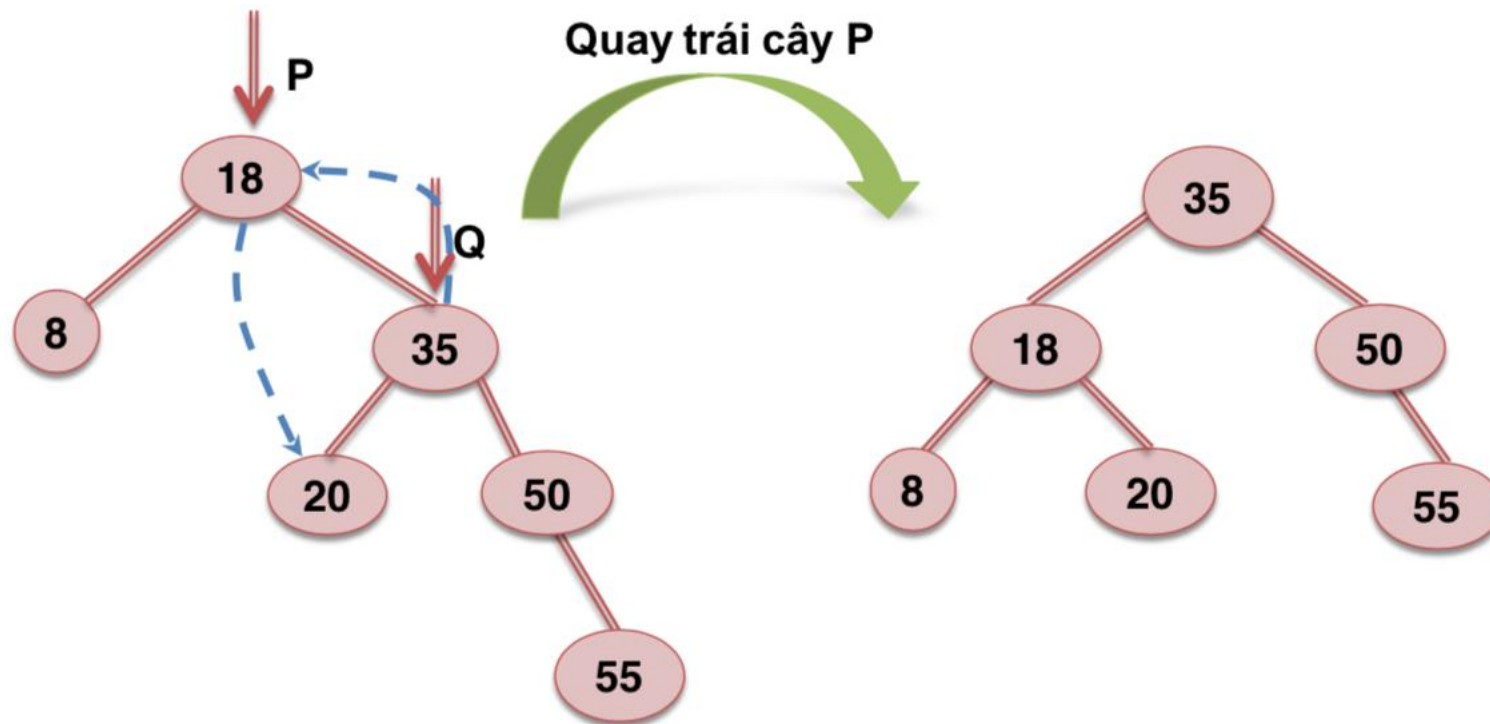
1.4.4. Xử lý mất cân bằng

- P mất cân bằng phải-phải (RR):



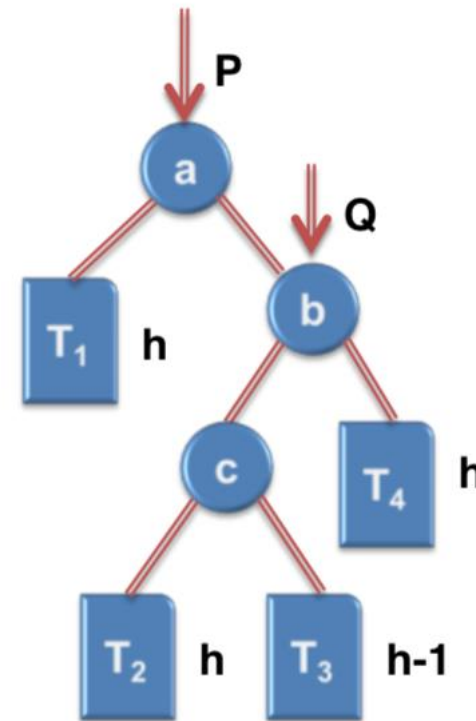
1.4.4. Xử lý mất cân bằng

- P mất cân bằng phải-phải (RR):



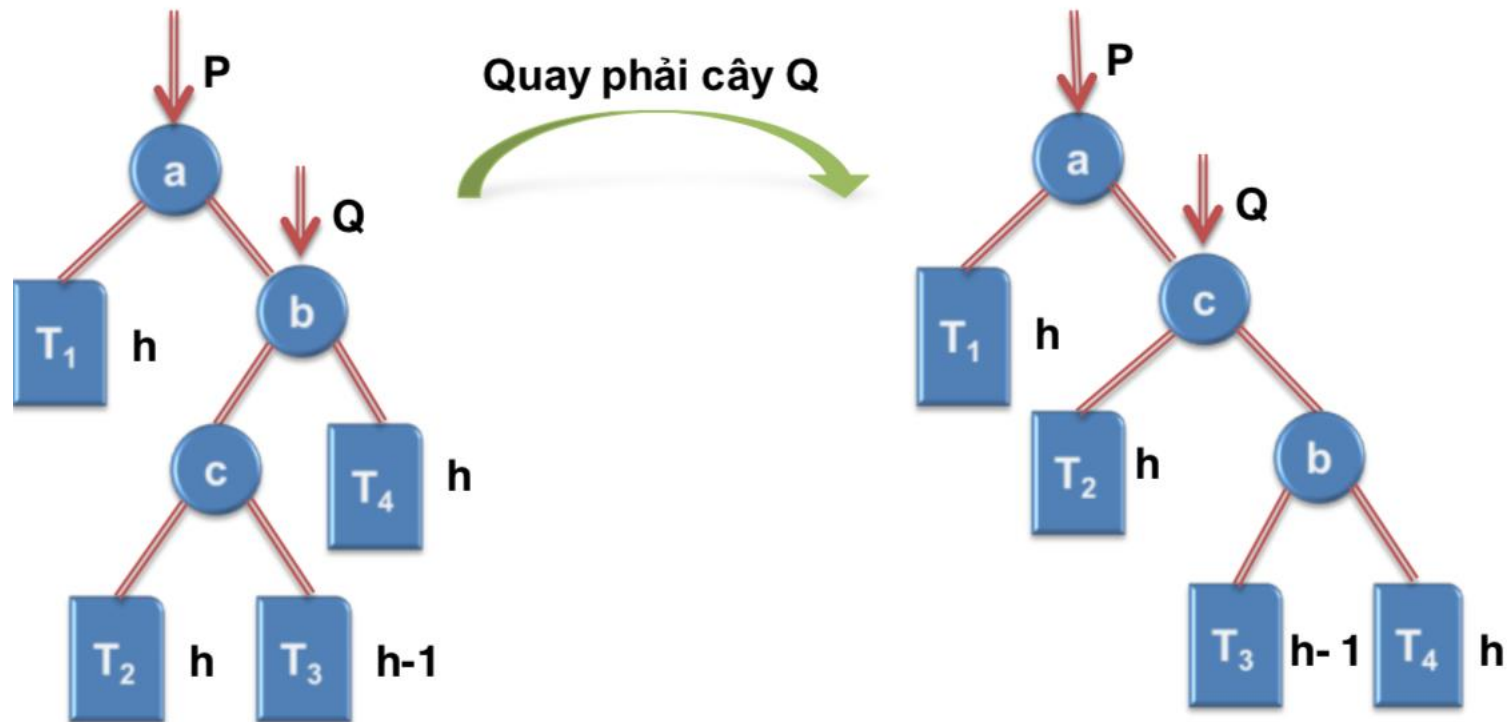
1.4.4. Xử lý mất cân bằng

- P mất cân bằng phải-trái (RL):
 - Bước 1: quay phải Q
 - Bước 2: quay trái cây P



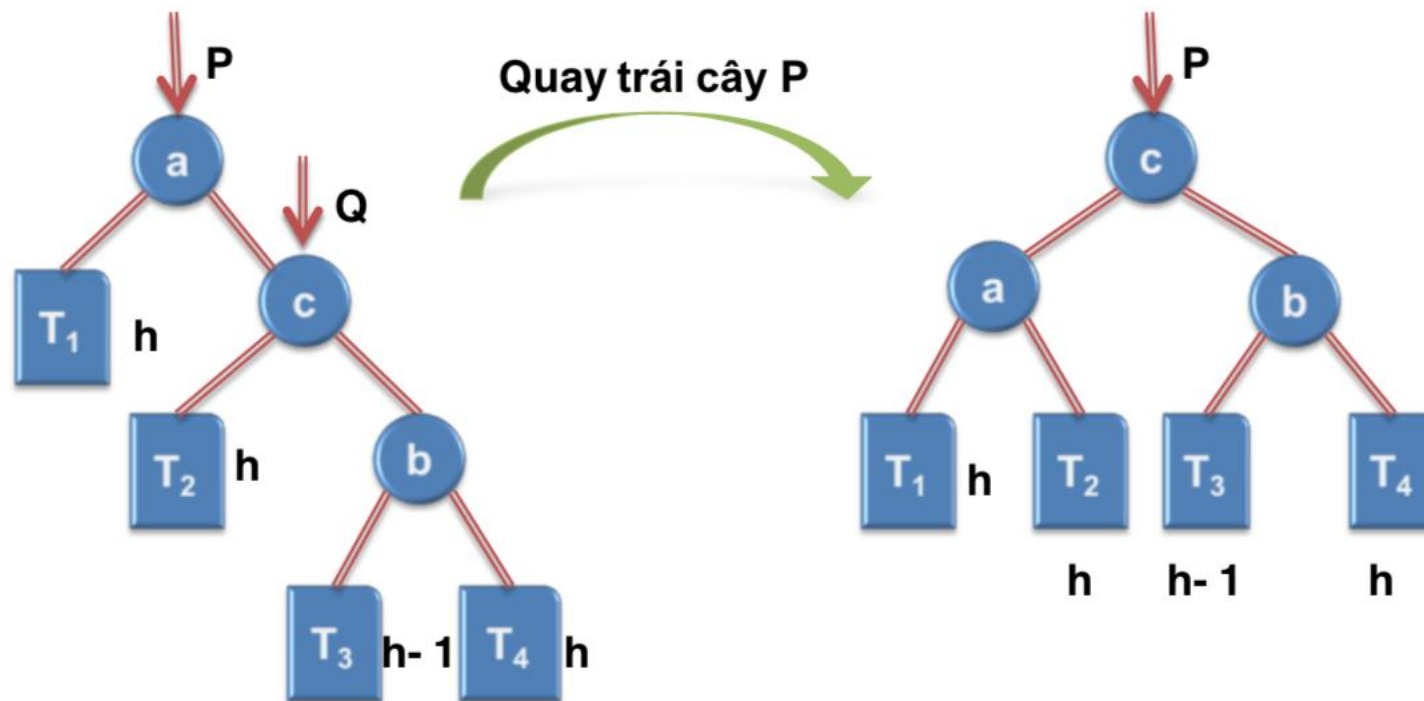
1.4.4. Xử lý mất cân bằng

- P mất cân bằng phải-trái (RL):
 - Bước 1: quay phải Q



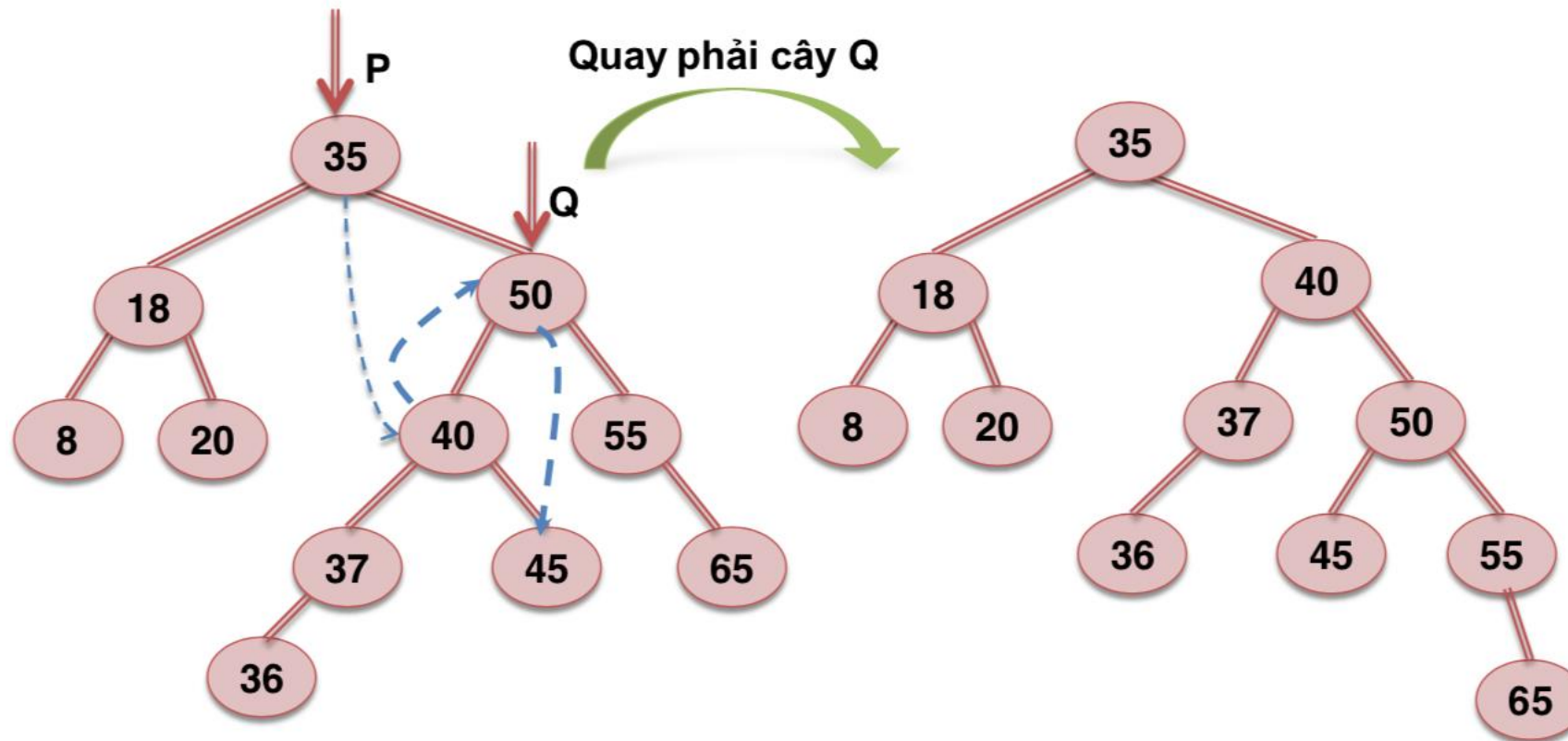
1.4.4. Xử lý mất cân bằng

- P mất cân bằng phải-trái (RL):
 - Bước 2: quay trái cây P



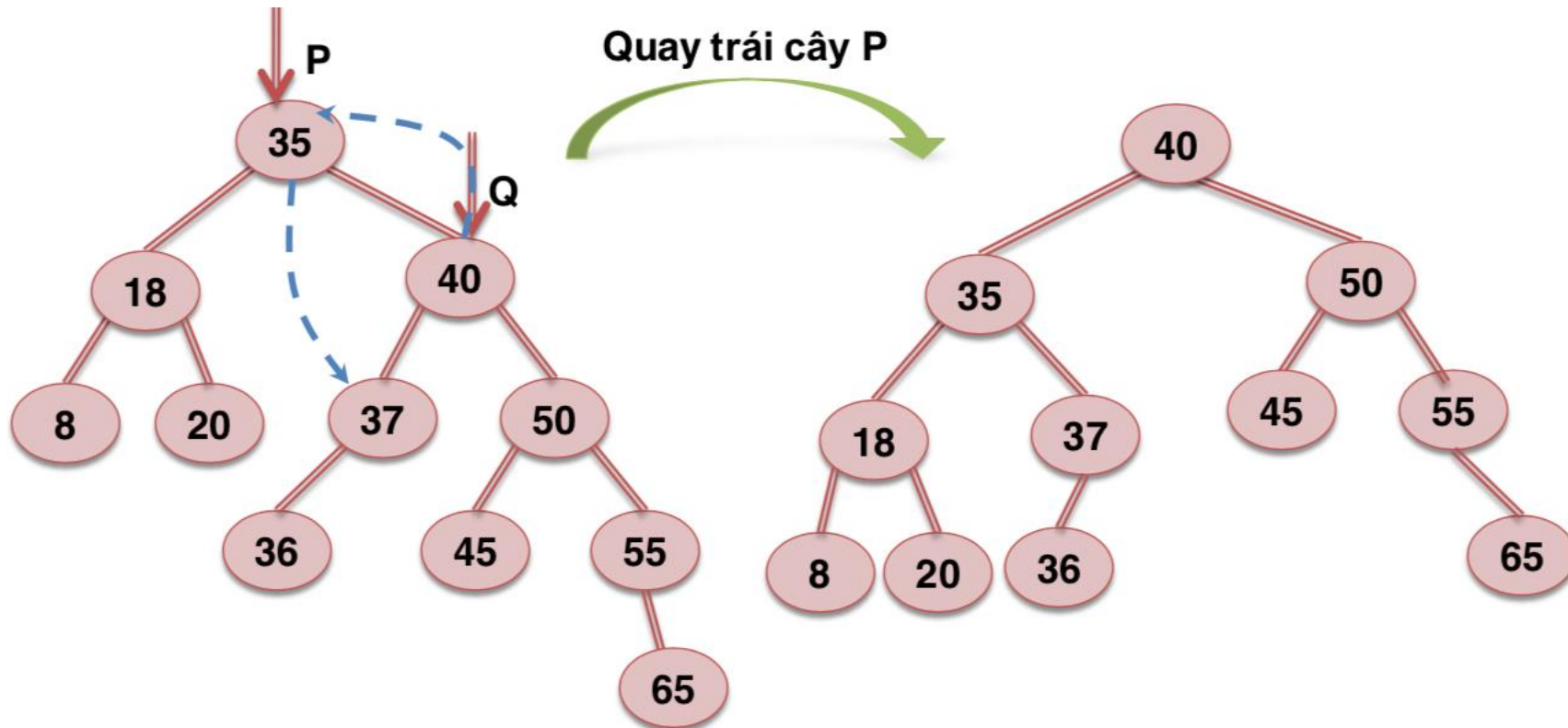
1.4.4. Xử lý mất cân bằng

- P mất cân bằng phải-trái (RL) - Bước 1:



1.4.4. Xử lý mất cân bằng

- P mất cân bằng phải-trái (RL) - Bước 2:



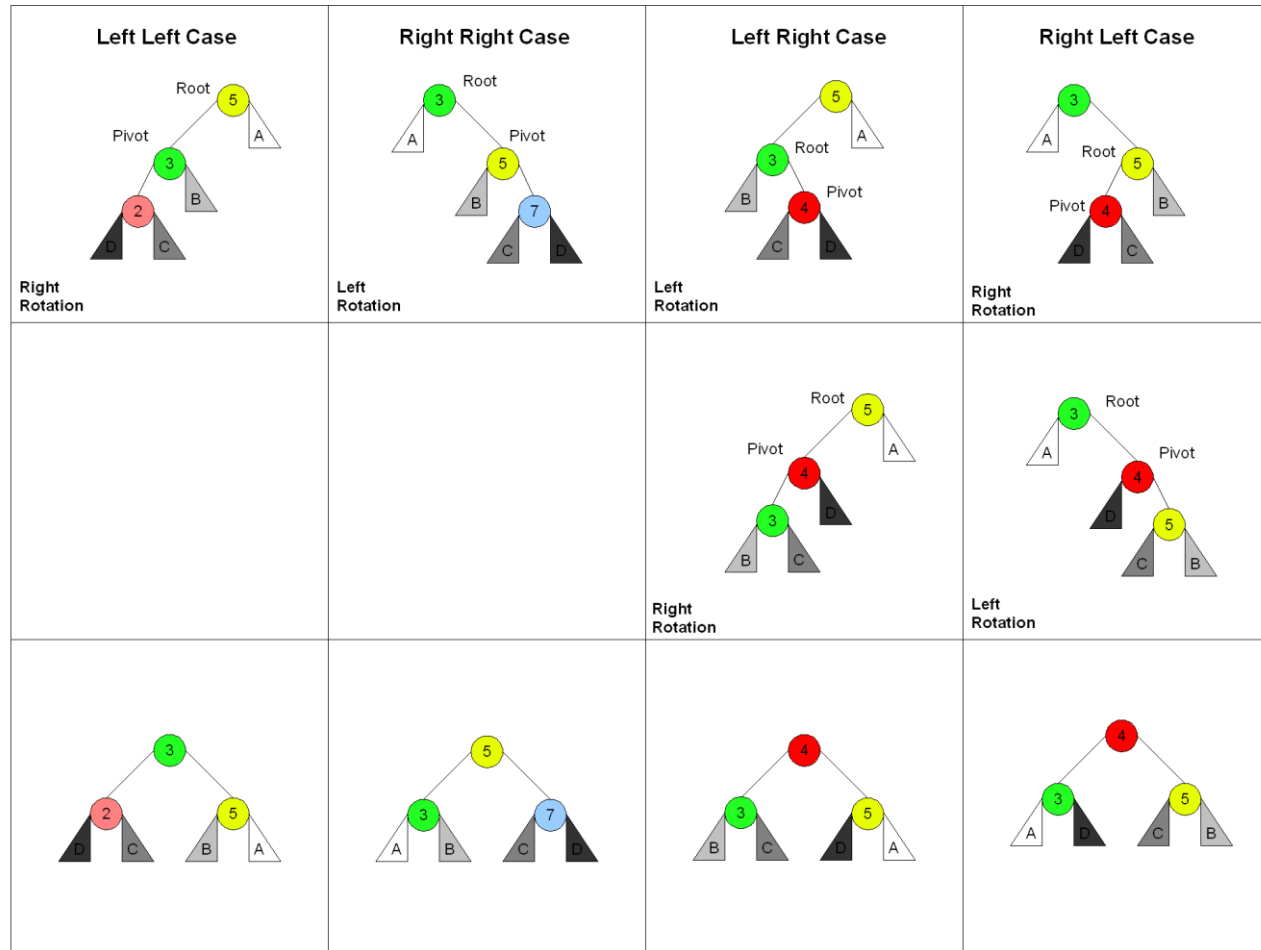
1.4.4. Xử lý mất cân bằng

- Khi một node cây xảy ra mất cân bằng bên trái (cây con trái chênh lệch với cây con phải hơn một đơn vị): (thực hiện đối xứng với trường hợp mất cân bằng bên phải)
 - Mất cân bằng trái-trái (LL)
 - Quay phải
 - Mất cân bằng trái-phải (L-R)
 - Quay trái
 - Quay phải

1.4.4. Xử lý mất cân bằng

There are 4 cases in all, choosing which one is made by seeing the direction of the first 2 nodes from the unbalanced node to the newly inserted node and matching them to the top most row.

Root is the initial parent before a rotation and **Pivot** is the child to take the root's place.

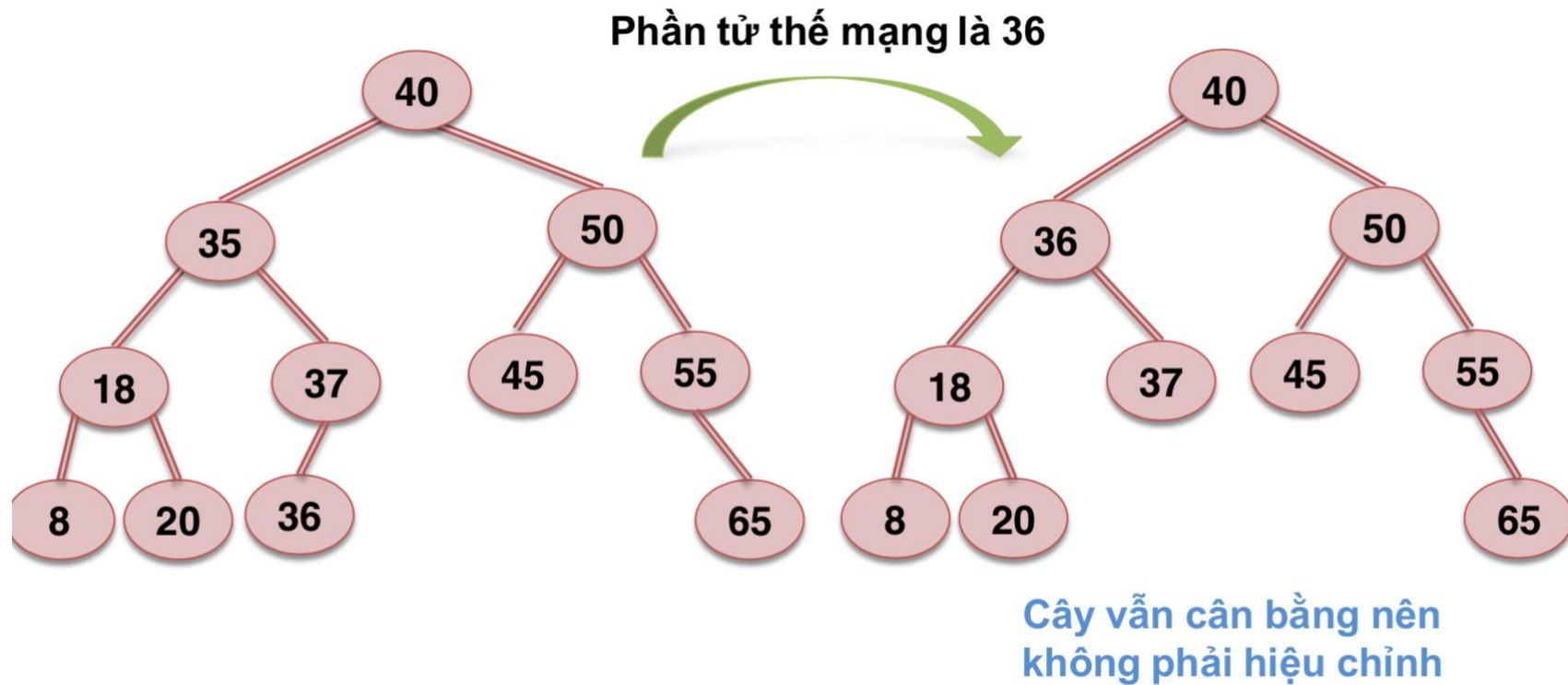


1.4.5. Các thao tác

- Thao tác tìm kiếm:
 - Thực hiện hoàn toàn tương tự cây nhị phân tìm kiếm.
- Thao tác thêm phần tử:
 - Thực hiện tương tự với việc thêm phần tử của cây nhị phân tìm kiếm.
 - Nếu xảy ra việc mất cân bằng thì xử lý bằng các trường hợp mất cân bằng đã biết.
- Thao tác xóa phần tử
 - Thực hiện tương tự cây nhị phân tìm kiếm: xét 3 trường hợp, và tìm phần tử thế mạng nếu cần.
 - Sau khi xóa, nếu cây mất cân bằng, thực hiện cân bằng cây.
 - Lưu ý: việc cân bằng sau khi hủy có thể xảy ra dây chuyền.

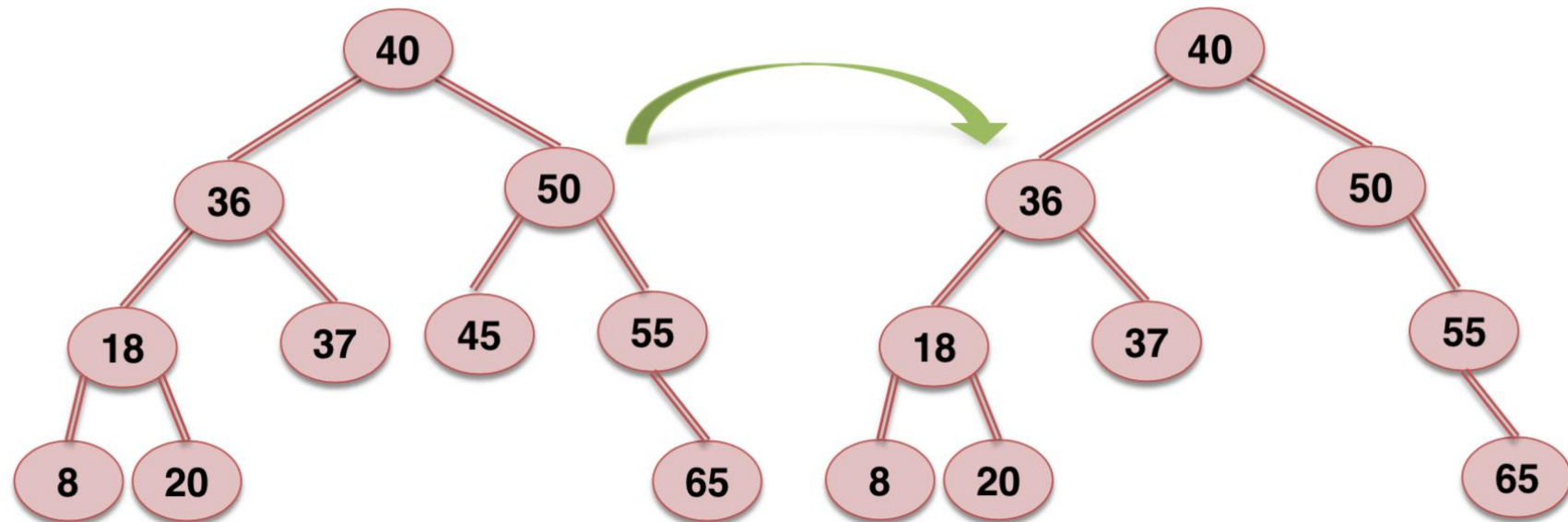
1.4.5. Các thao tác

- Ví dụ: xoá 35



1.4.5. Các thao tác

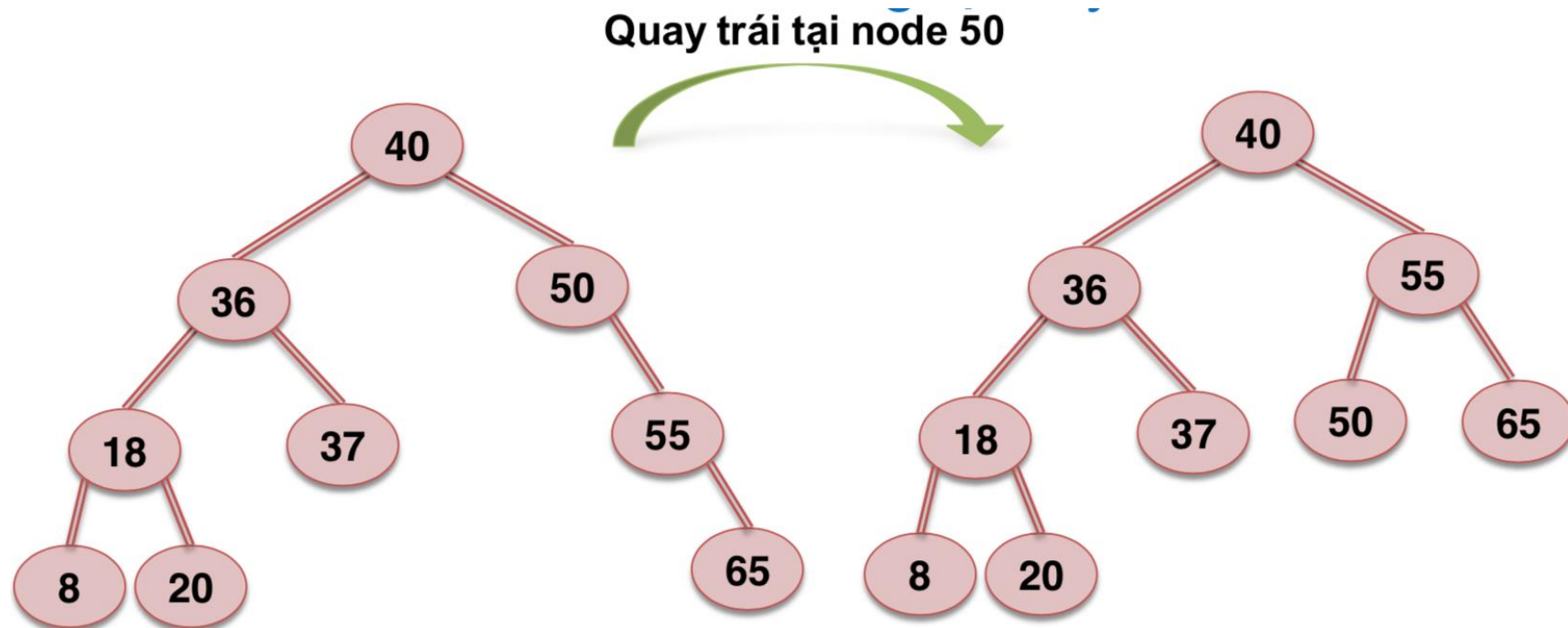
- Ví dụ: xoá 45



Node 50 bị lệch phải !!!

1.4.5. Các thao tác

- Ví dụ: xoá 45, cân bằng lại cây



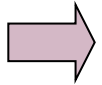
Chương 1. Tree

1.1. Các khái niệm cơ bản

1.2. Phép duyệt cây và biểu diễn cây

1.3. Cây nhị phân và cây nhị phân tìm kiếm

1.4. Cây AVL



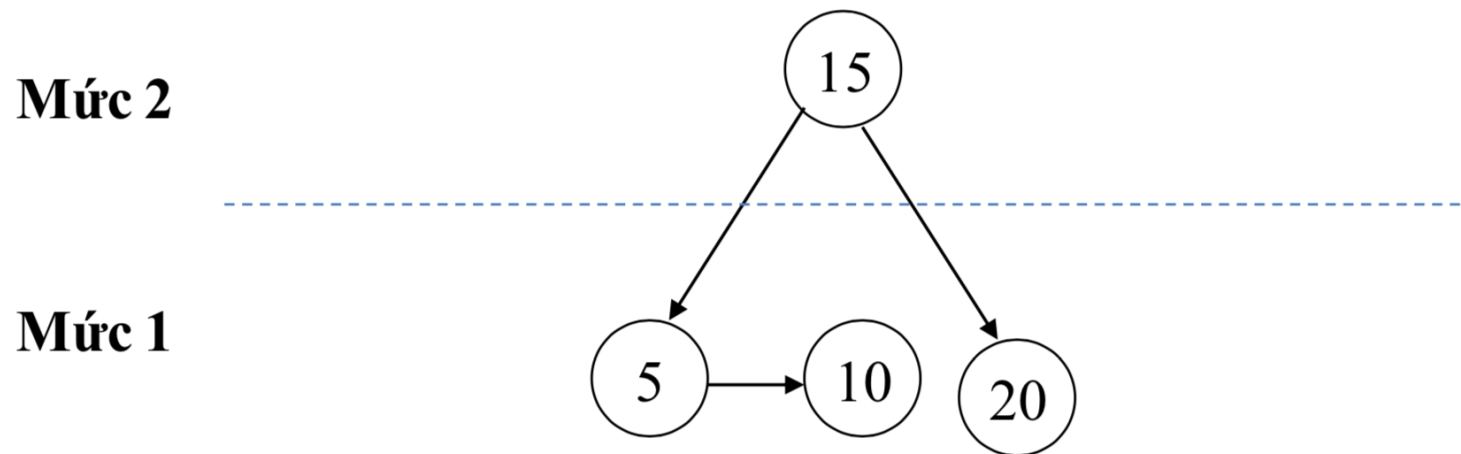
1.5. Cây AA

1.5. Cây AA

- Được đặt tên theo tác giả Arne Anderson (Thụy Điển).
- Công trình được công bố năm 1993 (Balanced Search Trees Made Simple).

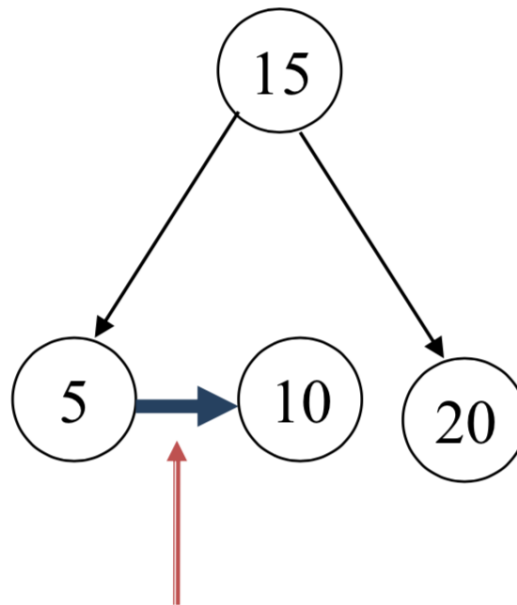
1.5.1. Các khái niệm

- Mức của node:
 - Số liên kết **trái** từ node đó đến node NULL.
 - Mức của NULL là 0
 - Mức của node lá là 1



1.5.1. Các khái niệm

- Liên kết ngang:
 - Liên kết giữa node cha và node con có cùng mức

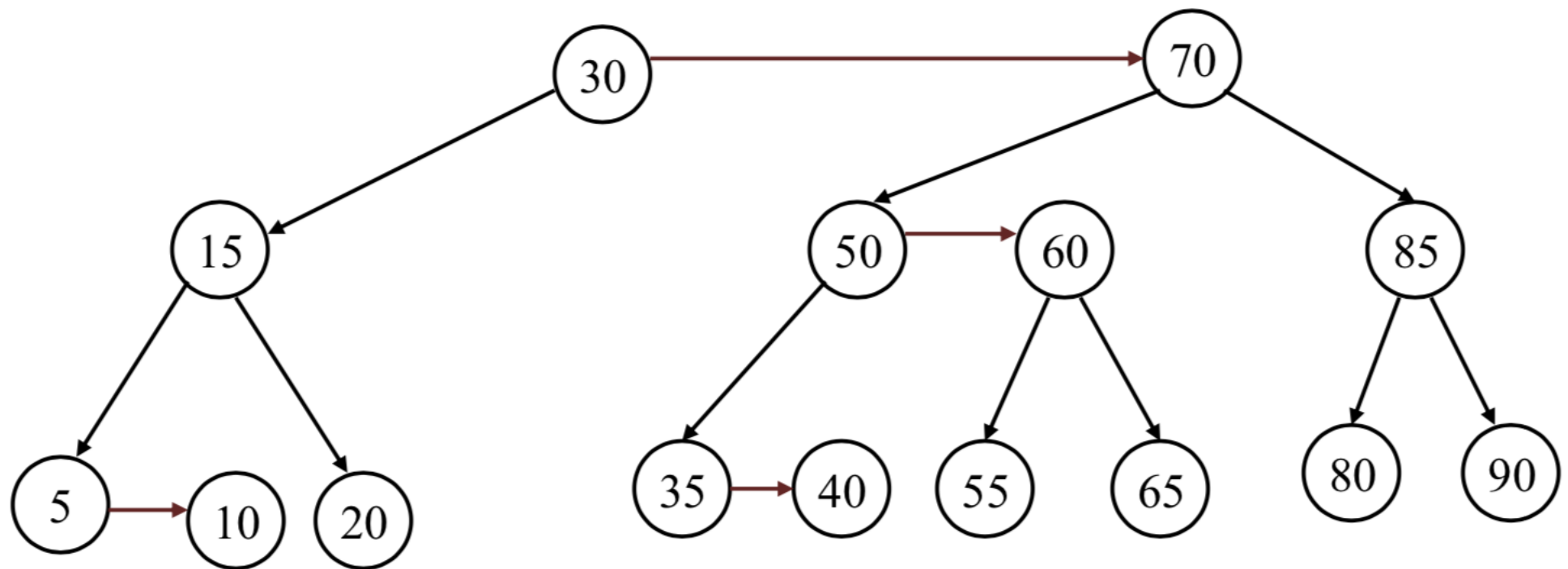


1.5.2. Tính chất

- Cây AA là cây nhị phân tìm kiếm thỏa mãn các tính chất sau:
 1. Mức của node con trái bắt buộc phải nhỏ hơn mức của node cha.
 2. Mức của node con bên phải nhỏ hơn hoặc bằng mức của node cha.
Liên kết ngang bắt buộc hướng sang phải.
 3. Mức của node cháu bên phải bắt buộc nhỏ hơn mức của node ông.
Không tồn tại 2 liên kết ngang liên tiếp.
 4. Mọi node có mức lớn hơn 1 phải có 2 node con.
 5. Nếu một node không có liên kết ngang phải thì cả hai node con của nó phải cùng mức.

1.5.2. Tính chất

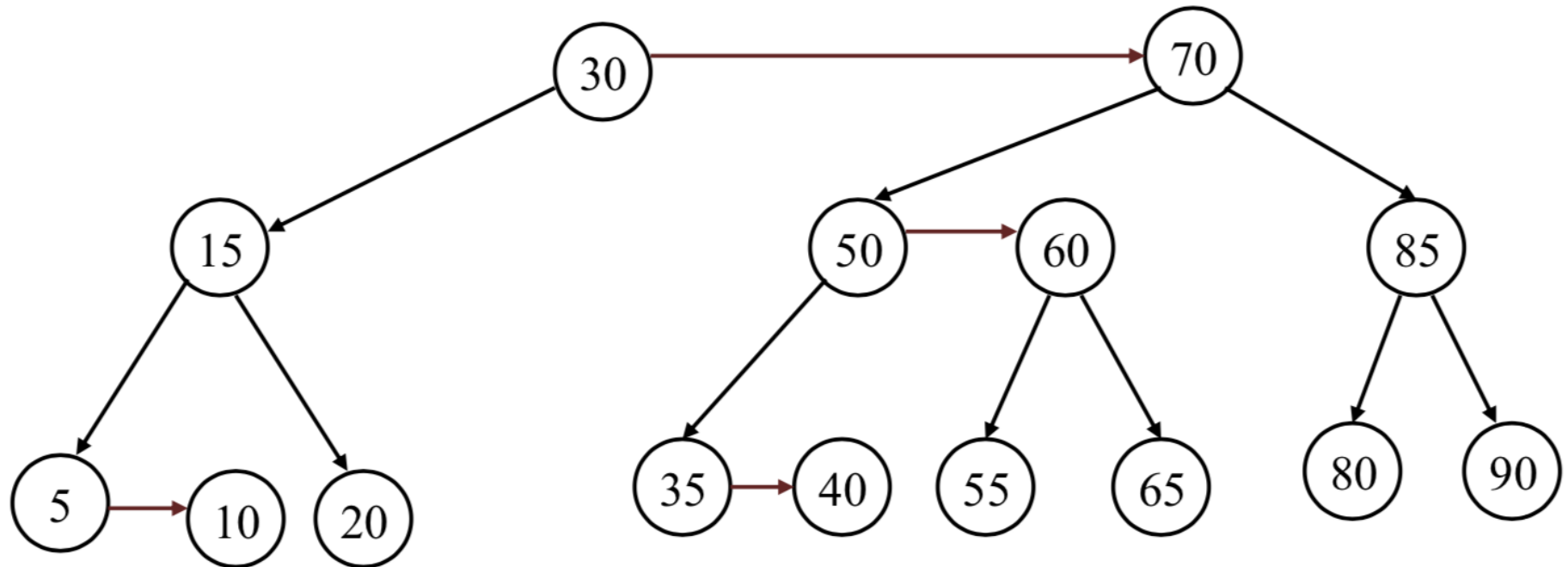
- Ví dụ



Mức của các node?

1.5.2. Tính chất

- Ví dụ

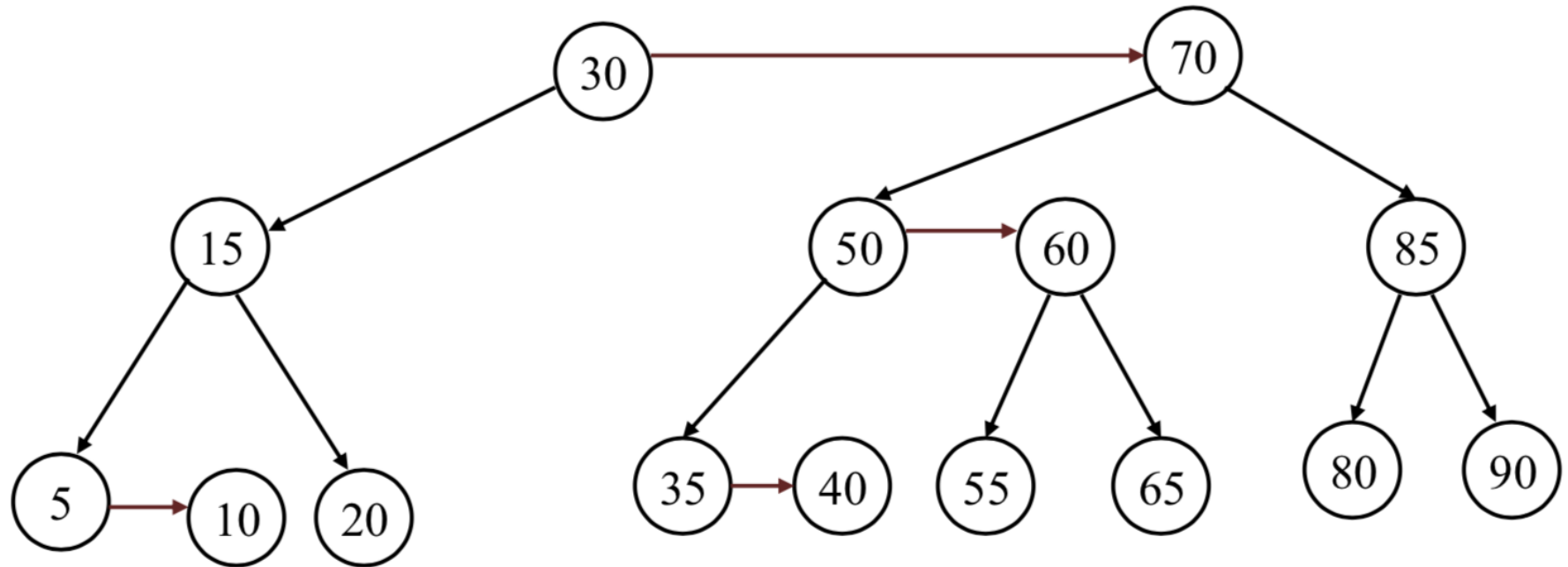


So sánh mức của node con trái với mức của node cha trực tiếp của nó?

Các cặp node: 15 và 30, 5 và 15, 50 và 70, 35 và 50, 55 và 60, 80 và 85

1.5.2. Tính chất

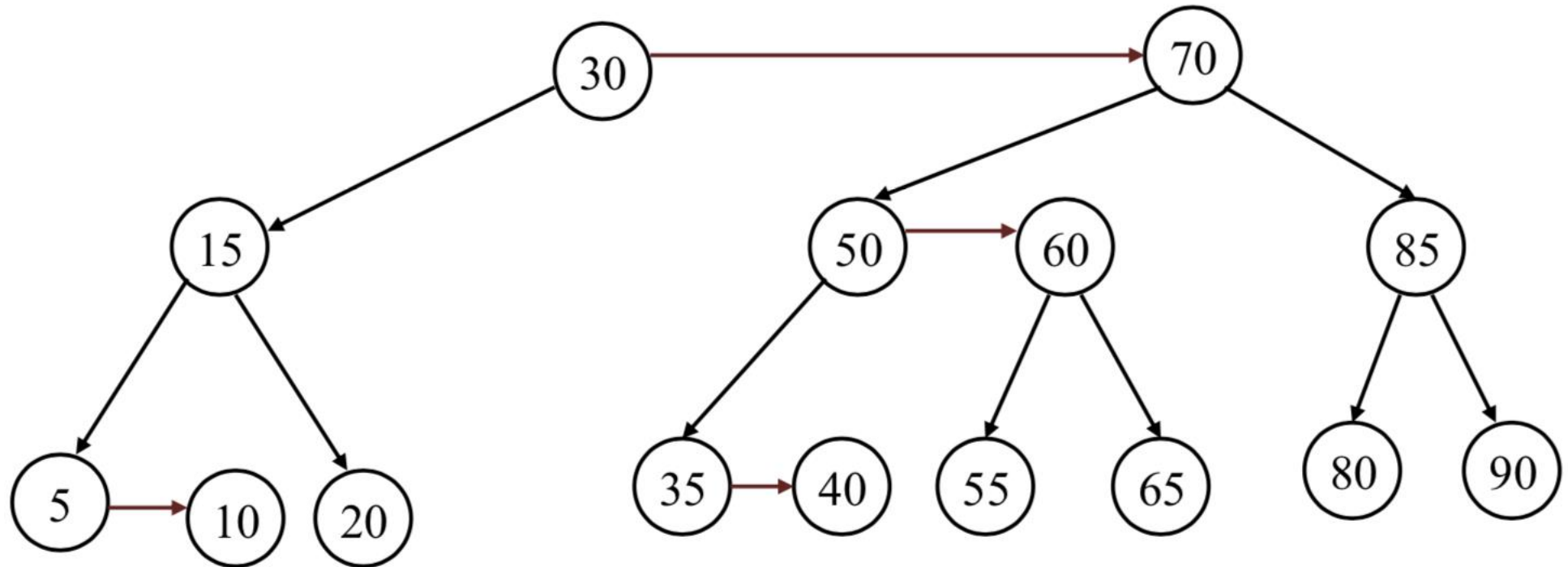
- Ví dụ



Các liên kết ngang?
Hướng của liên kết ngang?

1.5.2. Tính chất

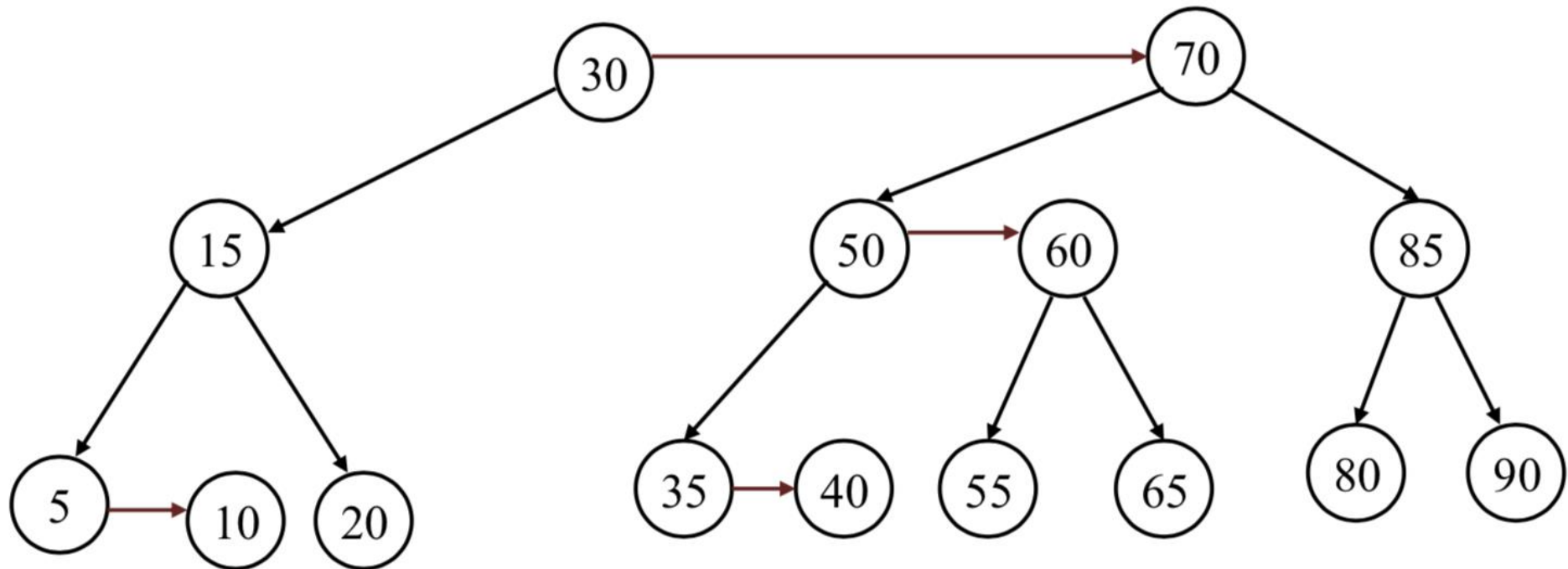
- Ví dụ



Có tồn tại 2 liên kết ngang liên tiếp?

1.5.2. Tính chất

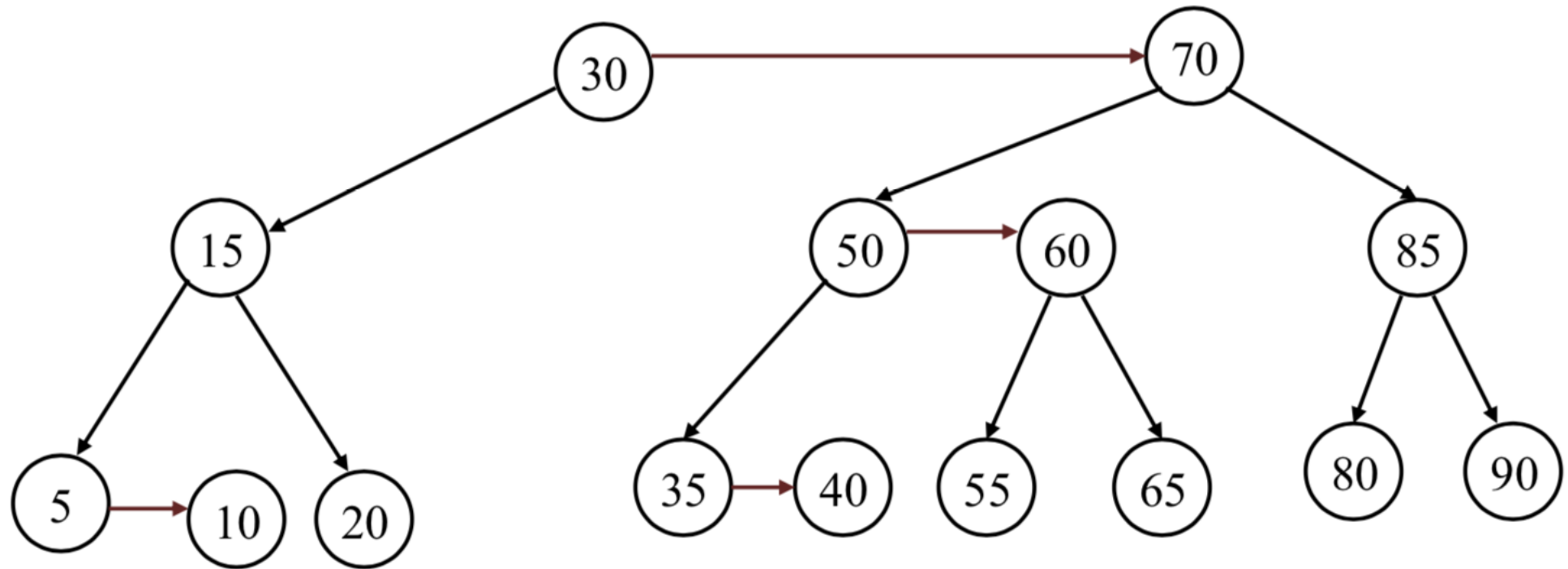
- Ví dụ



Mọi node có mức lớn hơn 1 đều có 2 node con?

1.5.2. Tính chất

- Ví dụ



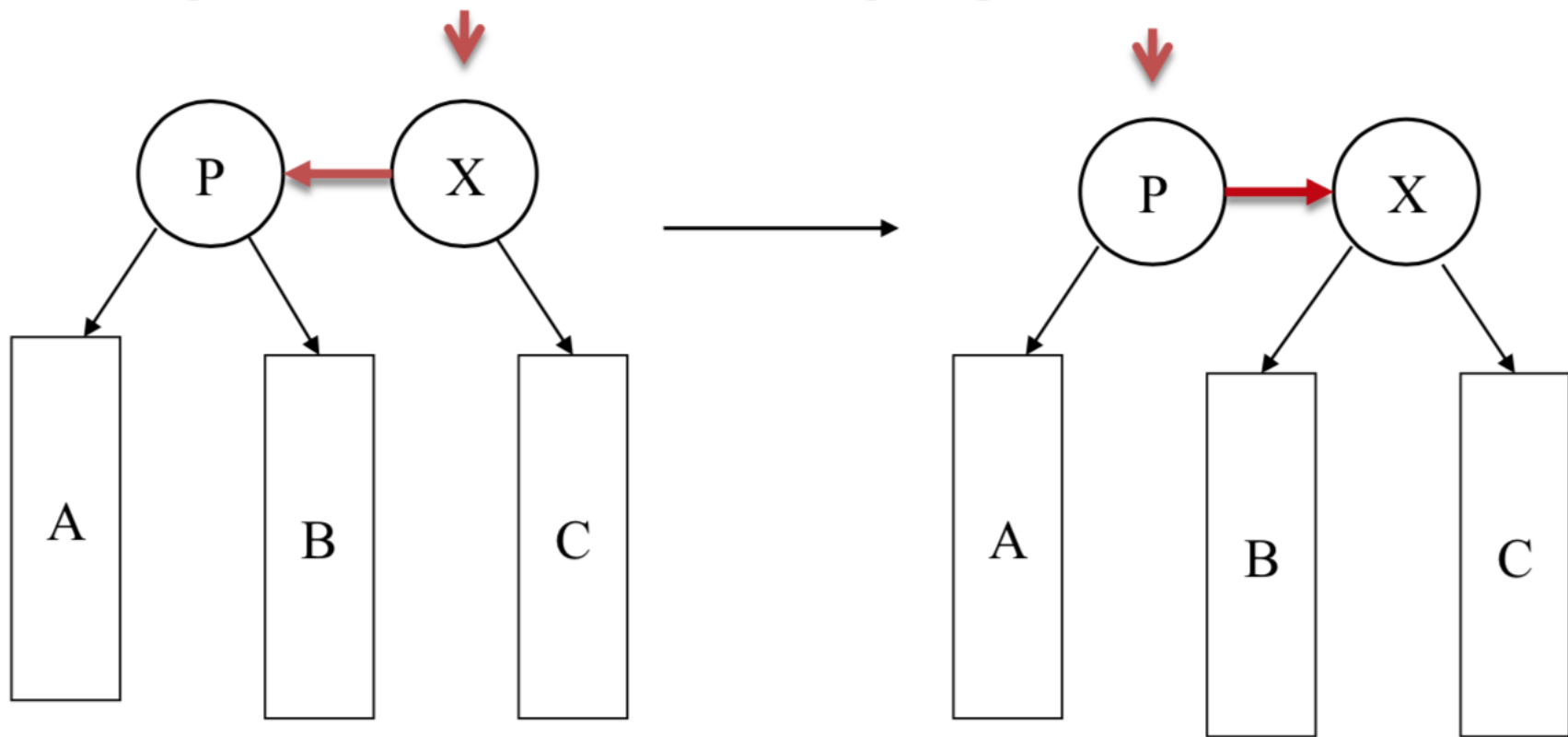
So sánh mức của các node con của các node: 15, 70, 60, 85?

1.5.3. Các phép biến đổi cây

- Skew
- Split

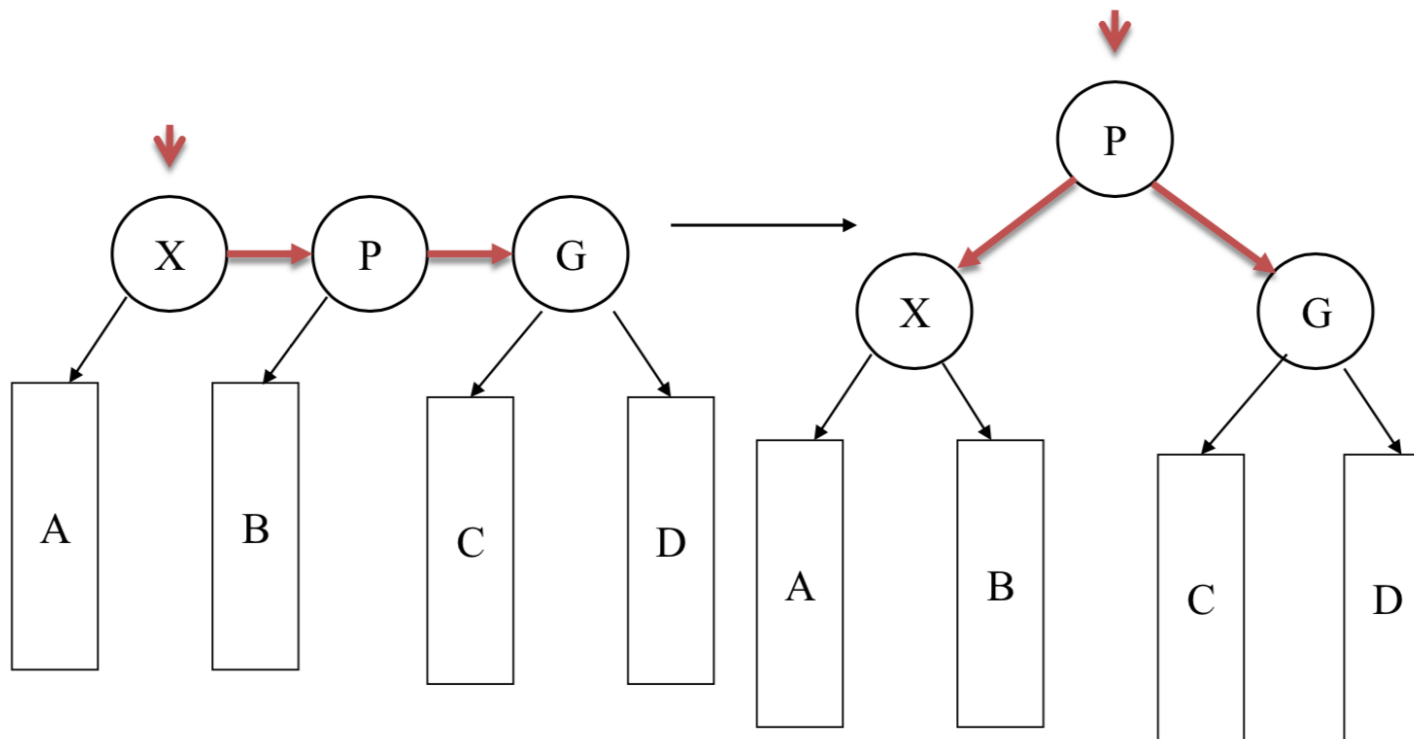
1.5.3. Các phép biến đổi cây

- Skew
 - Dùng để loại bỏ liên kết ngang trái



1.5.3. Các phép biến đổi cây

- Split
 - Dùng để loại bỏ 2 liên kết ngang liên tiếp



1.5.3. Các phép biến đổi cây

- Skew: dùng để loại bỏ liên kết ngang bên trái.
- Split: dùng để loại bỏ 2 liên kết ngang (phải) liên tiếp.
- Biến đổi theo thứ tự Skew -> Split (nếu có).
- Khi thực hiện thao tác Split, node giữa được tăng thêm một mức.

1.5.4. Các thao tác trên cây

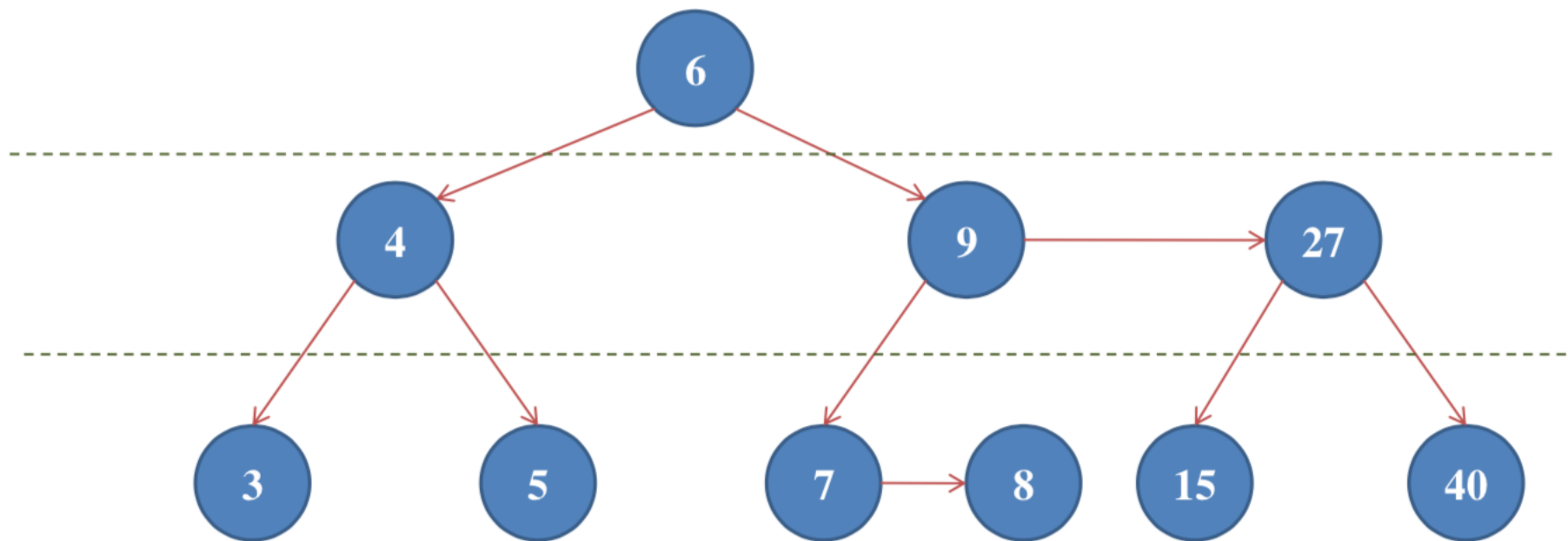
- Duyệt cây, Tìm kiếm:
 - Tương tự cây nhị phân tìm kiếm
- Thêm phần tử
- Xoá phần tử

1.5.4. Các thao tác trên cây

- Thêm phần tử:
 - Thực hiện tương tự trên cây nhị phân tìm kiếm.
 - Phần tử được thêm vào luôn ở mức 1.
 - Sau khi thêm, thực hiện các thao tác Skew và/hoặc Split để đảm bảo tính chất của cây.

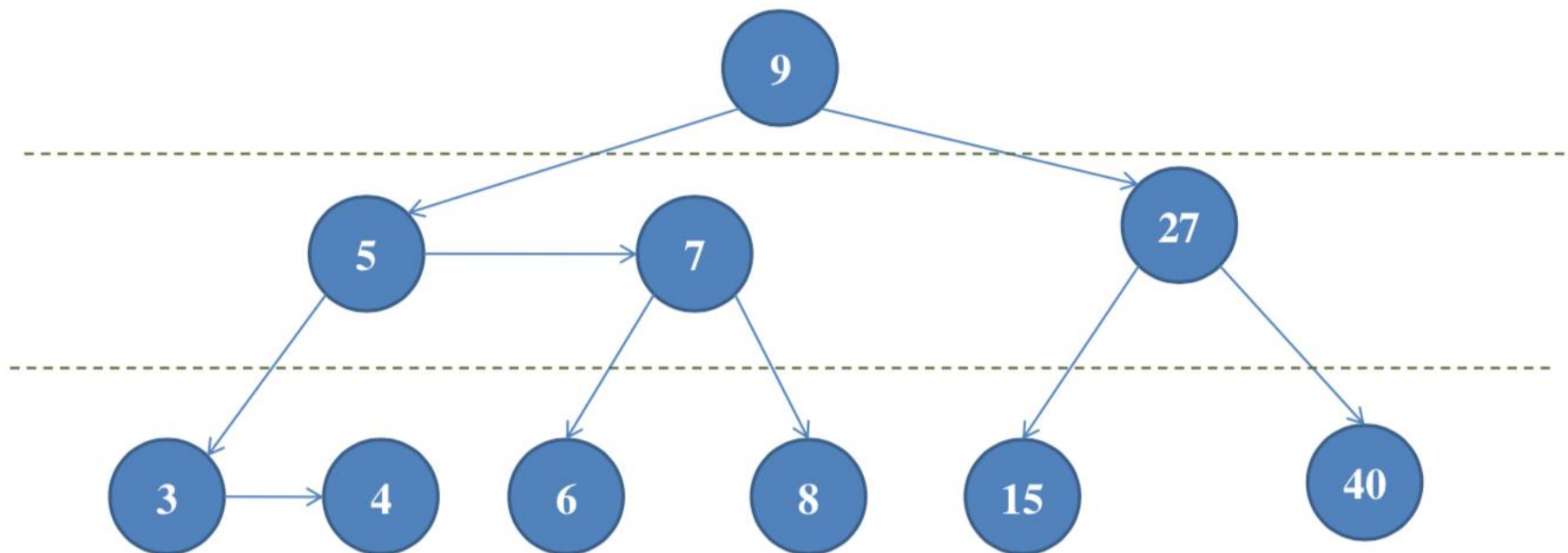
1.5.4. Các thao tác trên cây

- Ví dụ: Vẽ cây AA theo thứ tự nhập sau đây: 4, 7, 6, 3, 5, 9, 15, 27, 8, 40



1.5.4. Các thao tác trên cây

- Ví dụ: Hãy vẽ cây AA theo thứ tự nhập sau đây: 40, 8, 27, 15, 9, 5, 3, 6, 7, 4

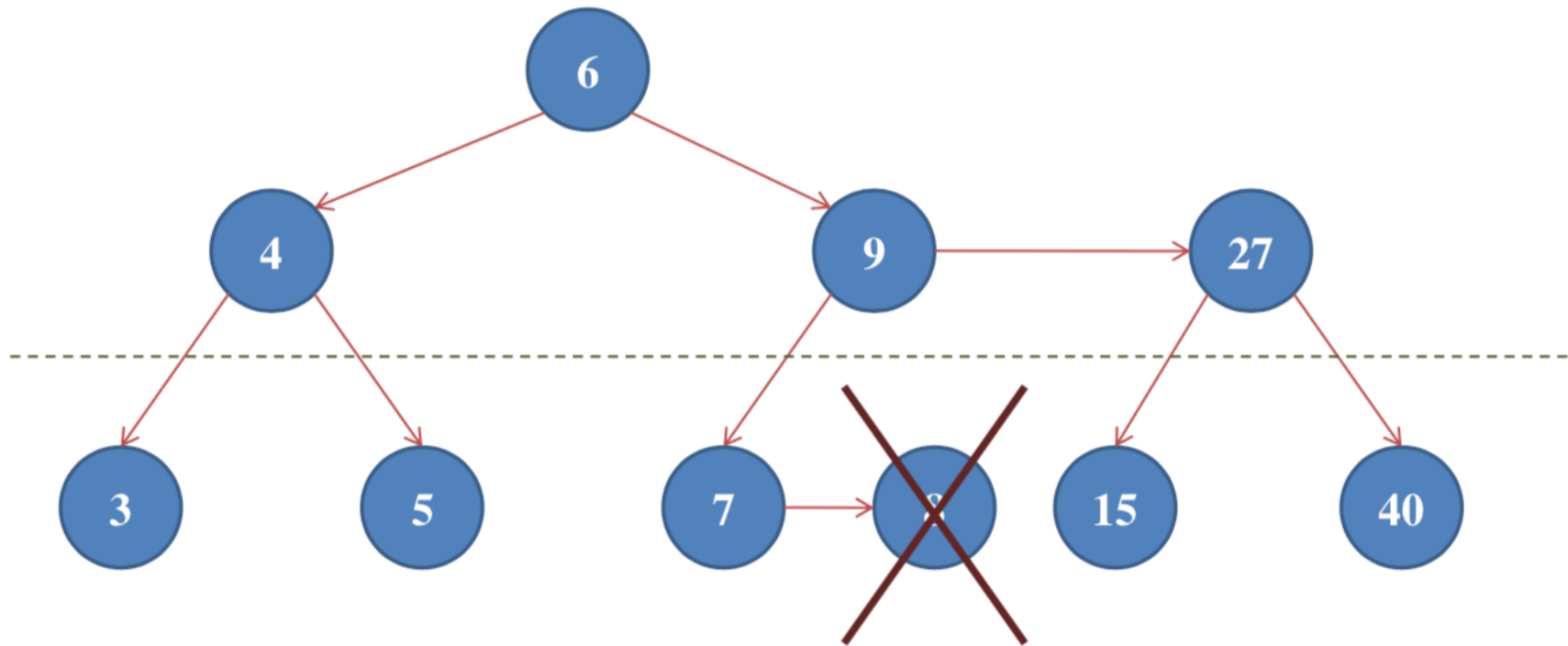


1.5.4. Các thao tác trên cây

- Xoá phần tử:
 - Nếu không phải là node lá (mức của node là 1), tìm phần tử thế mạng:
 - Phần tử lớn nhất bên nhánh trái (node lá).
 - Xóa node lá:
 - Giảm mức của node cha nếu mức của node lá nhỏ hơn.
 - Thực hiện các thao tác Skew, Split cần thiết

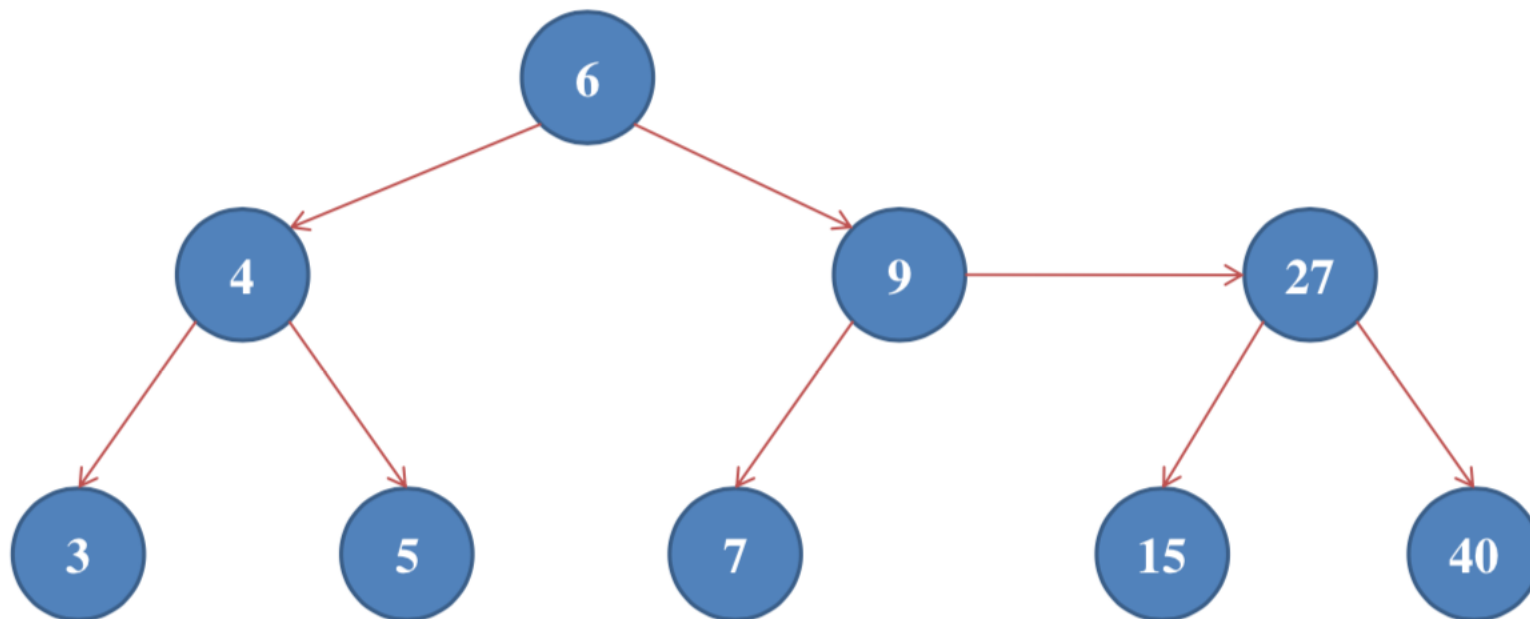
1.5.4. Các thao tác trên cây

- Ví dụ: Xoá phần tử 8



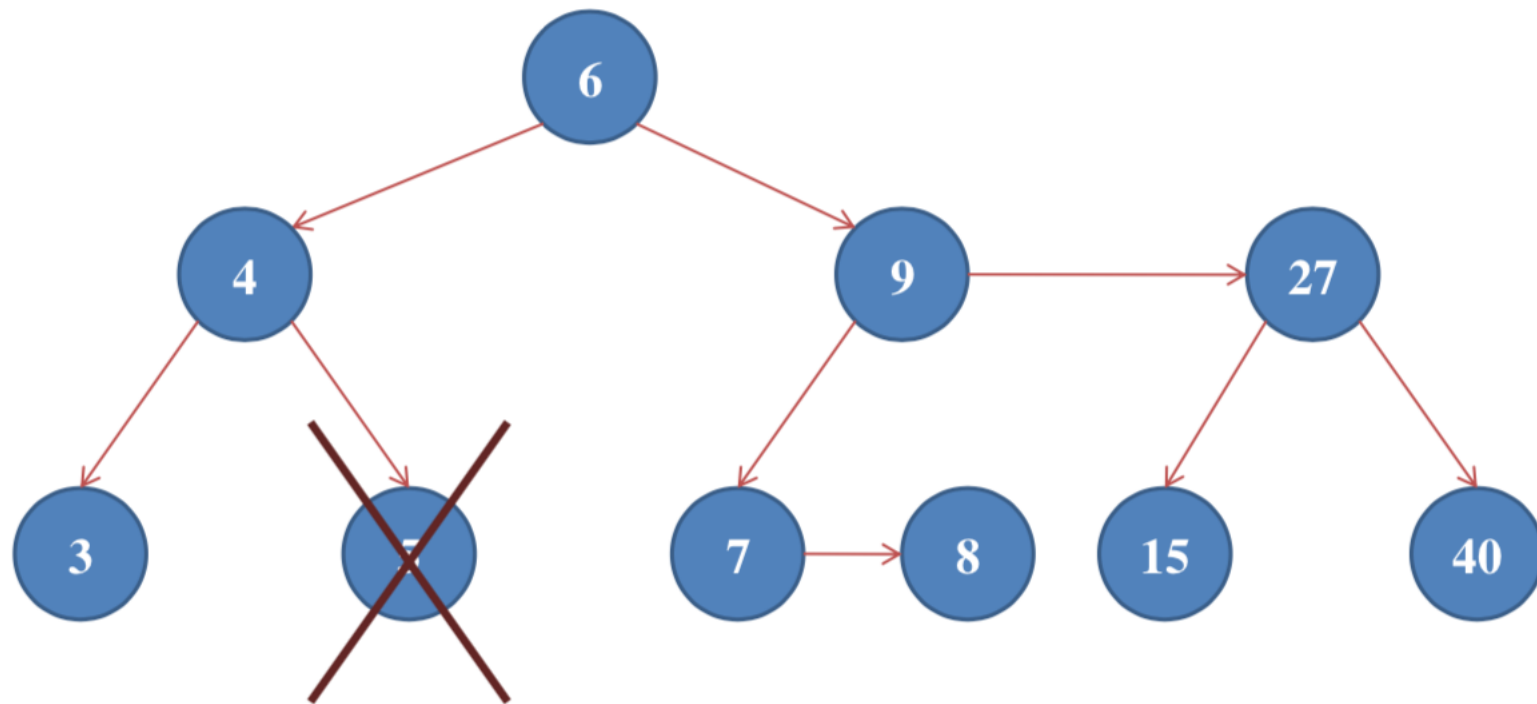
1.5.4. Các thao tác trên cây

- Ví dụ: Xoá phần tử 8



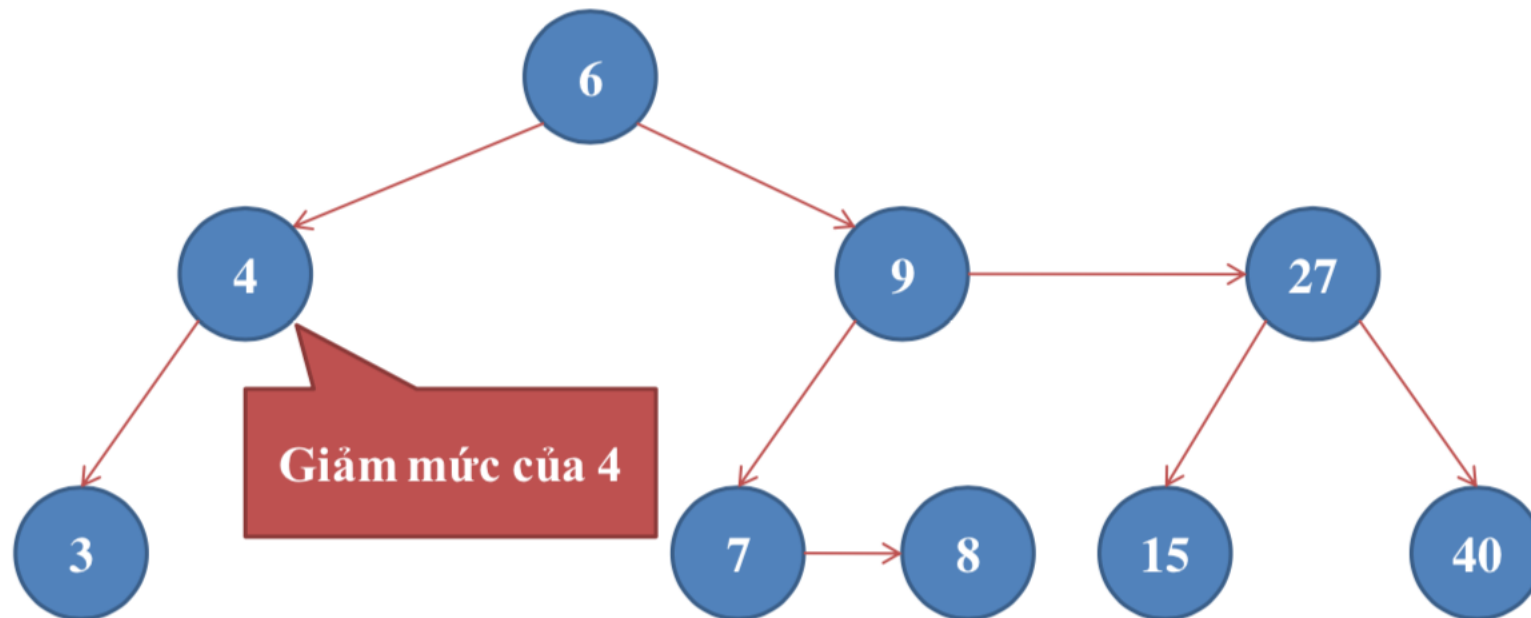
1.5.4. Các thao tác trên cây

- Ví dụ: Xoá phần tử 5



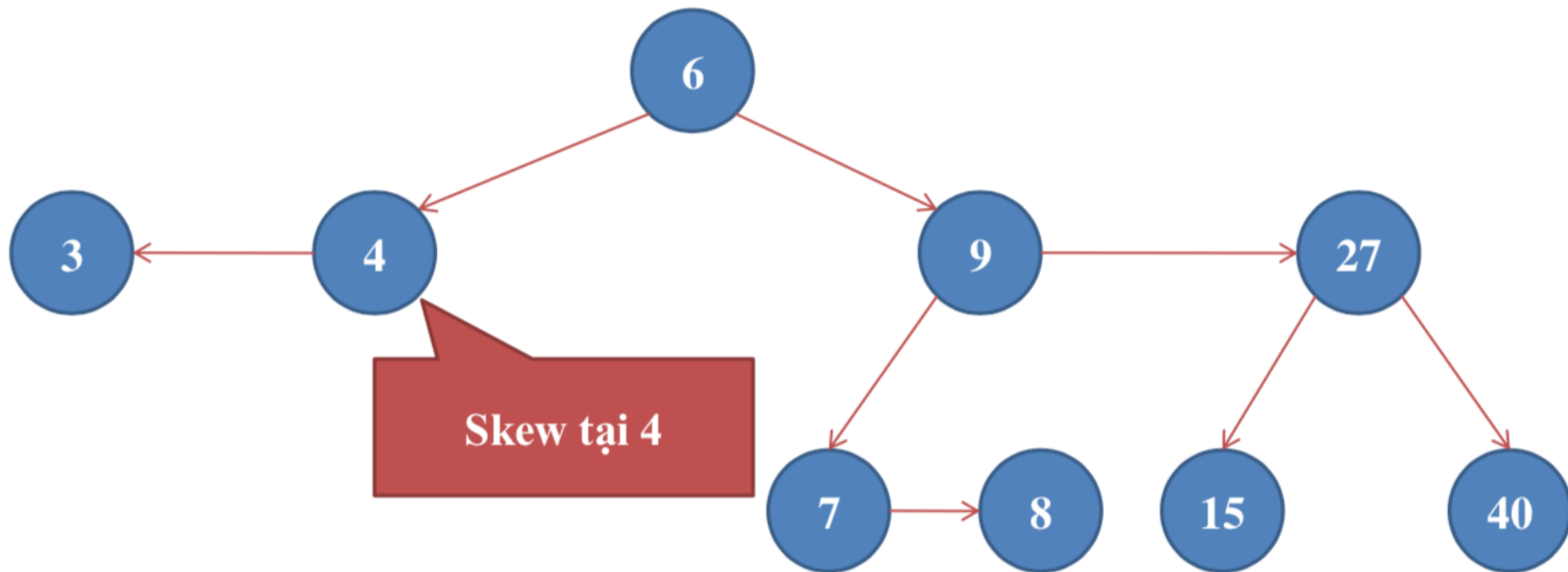
1.5.4. Các thao tác trên cây

- Ví dụ: Xoá phần tử 5



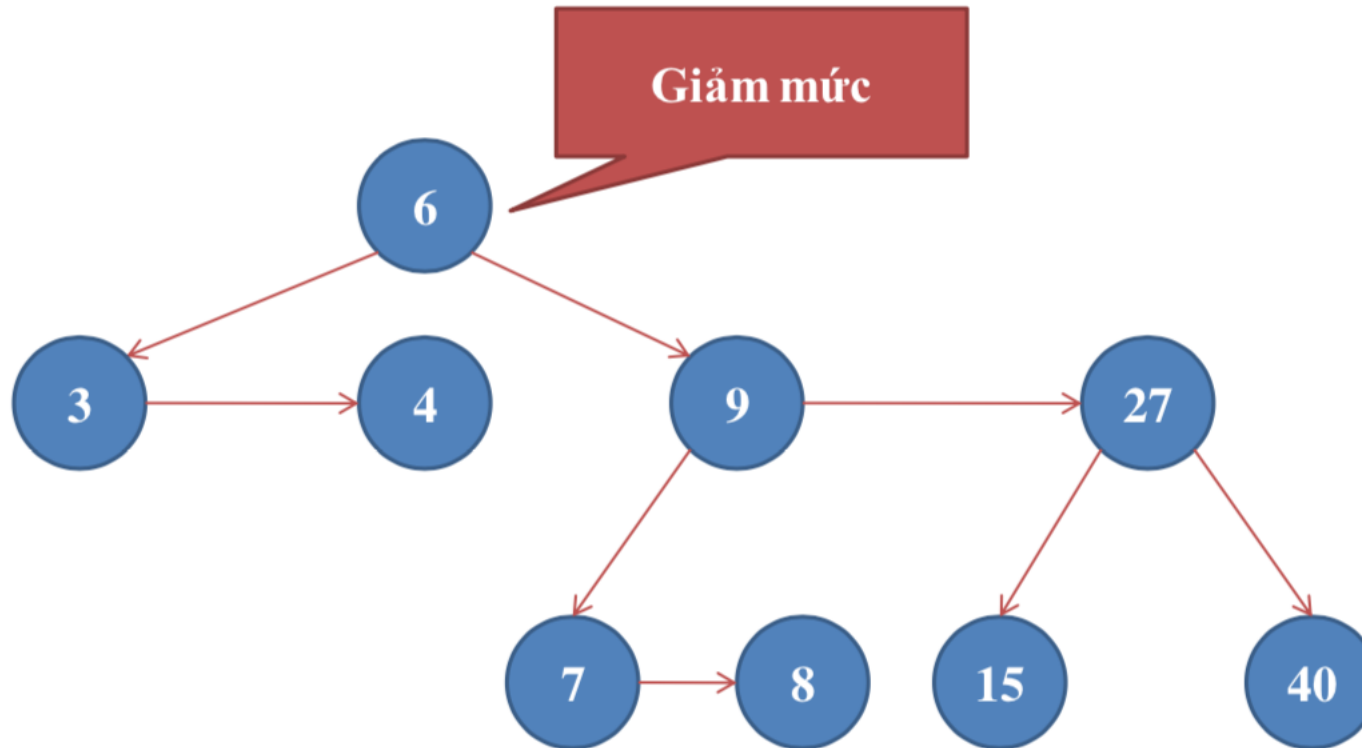
1.5.4. Các thao tác trên cây

- Ví dụ: Xoá phần tử 5



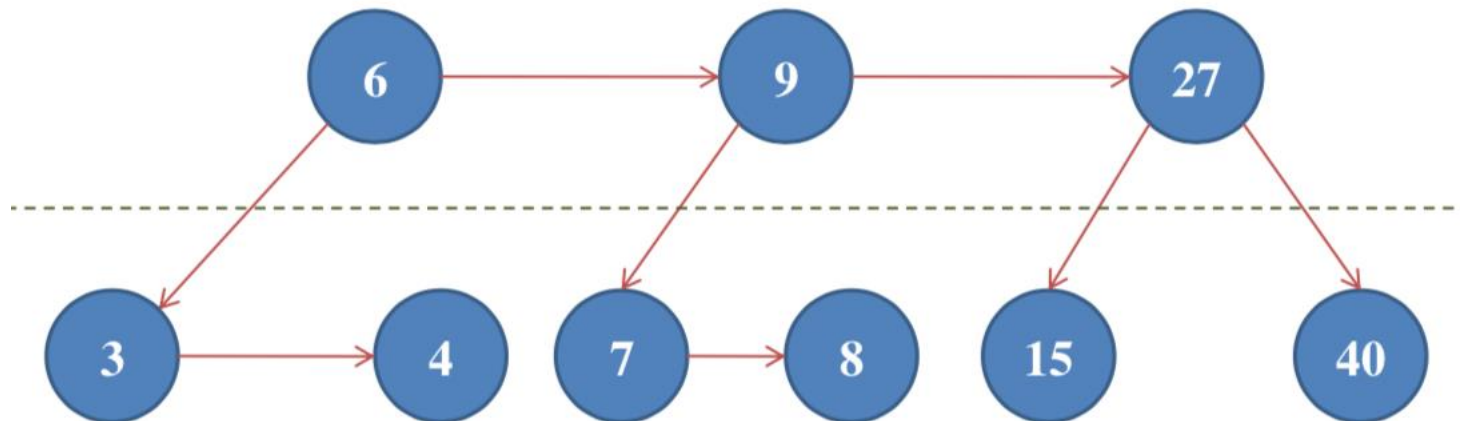
1.5.4. Các thao tác trên cây

- Ví dụ: Xoá phần tử 5



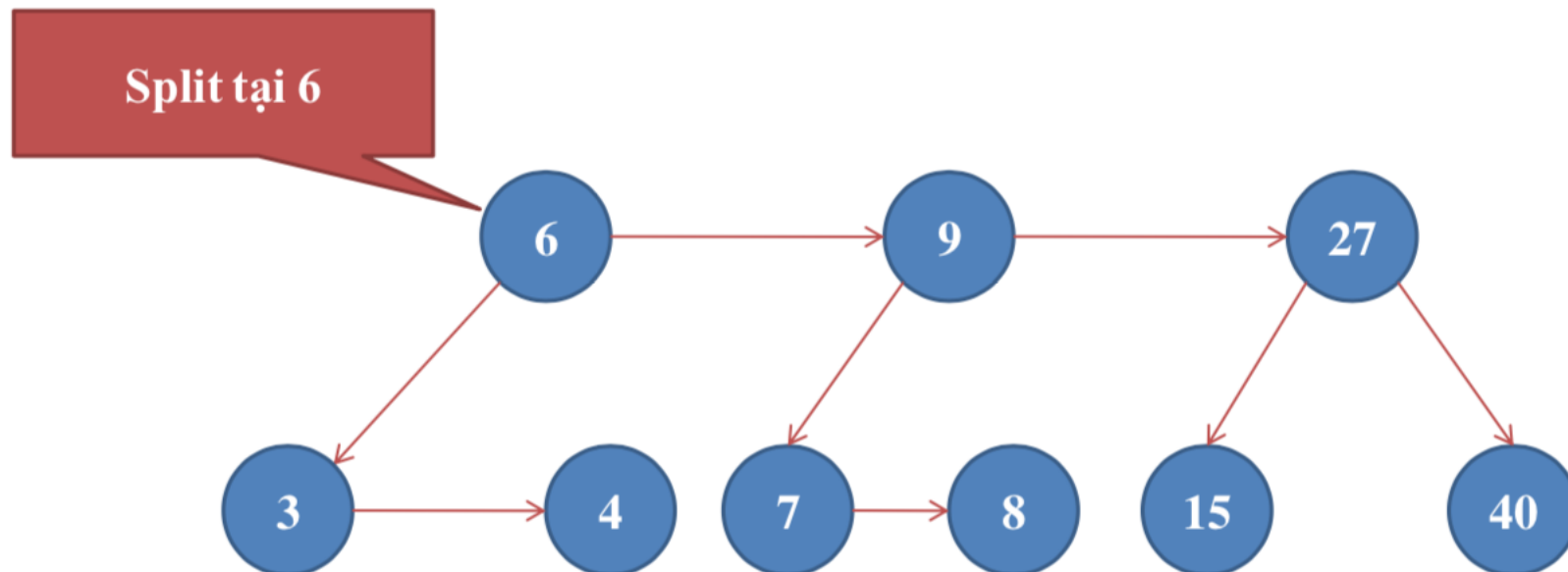
1.5.4. Các thao tác trên cây

- Ví dụ: Xoá phần tử 5



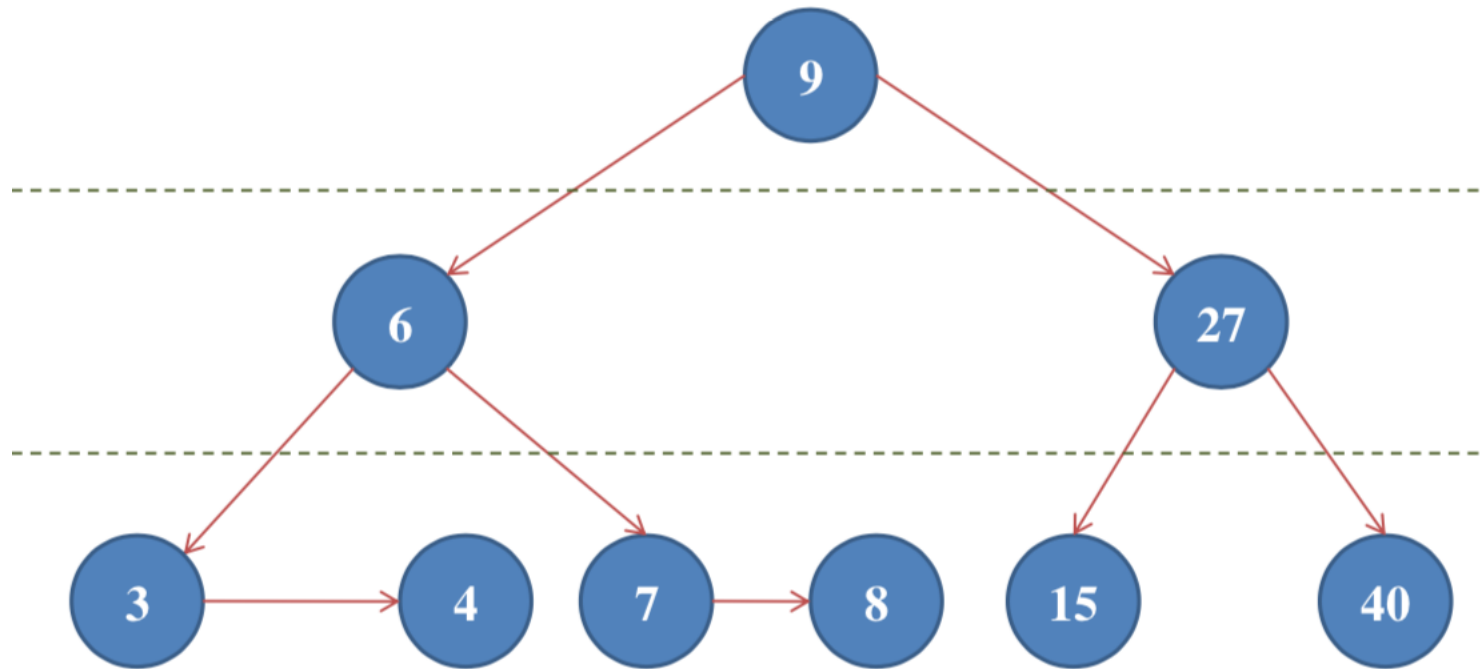
1.5.4. Các thao tác trên cây

- Ví dụ: Xoá phần tử 5



1.5.4. Các thao tác trên cây

- Ví dụ: Xoá phần tử 5



Câu hỏi

